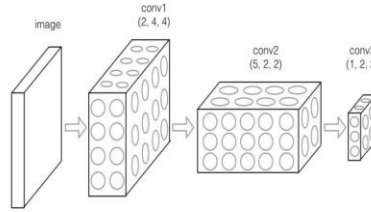
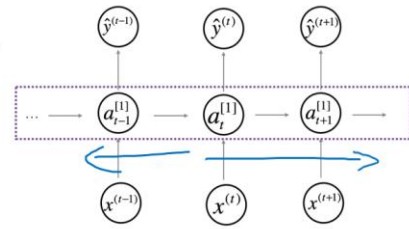


Standard NN



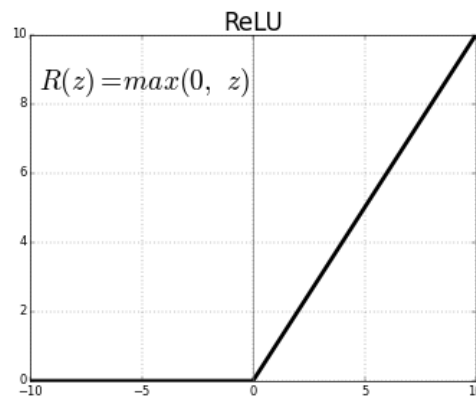
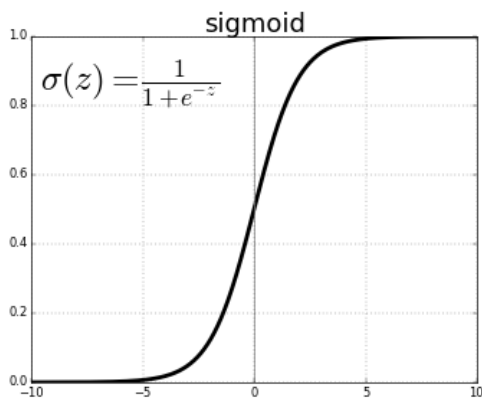
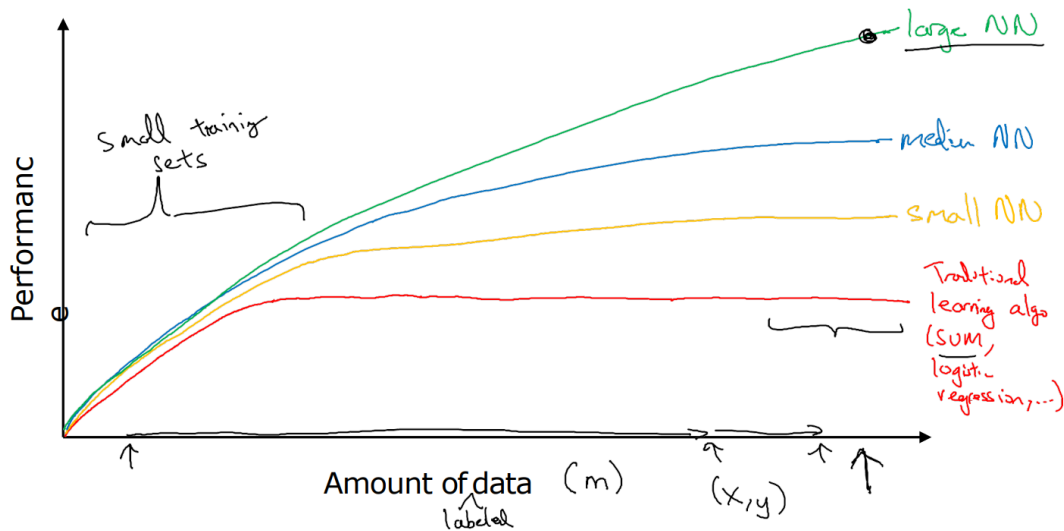
Convolutional NN



Recurrent NN

Structured data: rows and columns

Unstructured data: no rigid format (ex. audio, text, video, pixel values, words)



What does the analogy “AI is the new electricity” refer to?

Similar to electricity starting about 100 years ago, AI is transforming multiple industries.

Through the “smart grid”, AI is delivering a new wave of electricity.

AI runs on computers and is thus powered by electricity, but it is letting computers do things not possible before.

AI is powering personal devices in our homes and offices, similar to electricity.

Which of the following best describes the role of AI in the expression "an AI-powered society"?

AI controls the power grids for energy distribution, so all the power needed for industry and in daily life comes from AI.

AI is an essential ingredient in realizing tasks, in industry and in personal life.

AI helps to create a more efficient way of producing energy to power industries and personal devices.

Which of the following are reasons that didn't allow Deep Learning to be developed during the '80s?

People were afraid of a machine rebellion.

Interesting applications such as image recognition require large amounts of data that were not available.

The theoretical tools didn't exist during the 80's.

Limited computational power.

Which of these are reasons for Deep Learning recently taking off?

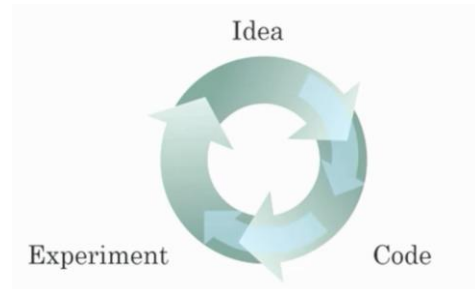
Deep learning has resulted in significant improvements in important applications such as online advertising, speech recognition, and image recognition.

Neural Networks are a brand new field.

We have access to a lot more data.

We have access to a lot more computational power.

Recall the diagram of iterating over different ML ideas. Which of the stages shown in the diagram was improved with the use of a better GPU/CPU?



With larger datasets, the iteration process is faster.

Without better hardware, there is no way to train models faster.

Some algorithms are specifically designed to run experiments faster.

Experiments finish faster, producing better ideas through increased iteration tempo.

When experienced deep learning engineers work on a new problem, they can usually use insight from previous problems to train a good model on the first try, without needing to iterate multiple times through different models. True/False?

True

False

Neural networks are good at figuring out functions relating an input x to an output y given enough examples. True/False?

False **True**

ReLU stands for which of the following?

Representation Linear Unit

Recognition Linear Unit

Rectified Linear Unit

Rectified Last Unit

Features of animals, such as weight, height, and color, are used for classification between cats, dogs, or others. This is an example of "structured" data, because they are represented as arrays in a computer. True/False?

False

No. The data can be represented by columns of data. This is an example of structured data, unlike images of the animal.

True

Yes. The data can be represented by columns of data. This is an example of structured data, unlike images of the animal.

A demographic dataset with statistics on different cities' population, GDP per capita, and economic growth is an example of "unstructured" data because it contains data coming from different sources. True/False?

True **False**

Which of the following are examples of unstructured data? Choose all that apply.

Sound files for speech recognition.

Images for bird recognition.

Information about elephants' weight, height, age, and the number of offspring.

Text describing size and number of pages of books.

Which of the following are examples of structured data? Choose all that apply.

A dataset with short poems.

A set of audio recordings of a person saying a single word.

A dataset with zip code, income, and name of a person.

A dataset of weight, height, age, the sugar level in the blood, and arterial pressure.

RNNs (Recurrent Neural Networks) are good for data with a temporal component.
True/False?

True

False

Why is an RNN (Recurrent Neural Network) used for machine translation, say translating English to French?

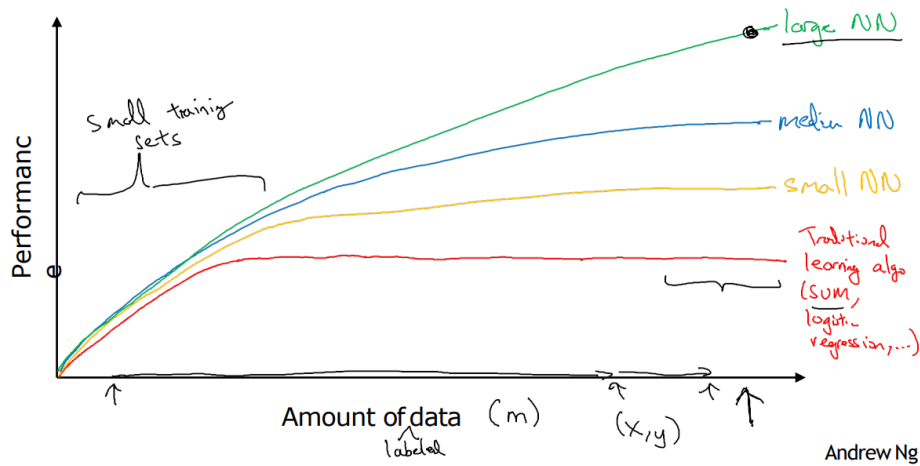
It is strictly more powerful than a Convolutional Neural Network (CNN).

It can be trained as a supervised learning problem.

It is applicable when the input/output is a sequence (e.g., a sequence of words).

RNNs represent the recurrent process of Idea->Code->Experiment->Idea->....

Scale drives deep learning progress



From the given diagram, we can deduce that Large NN models are always better than traditional learning algorithms. True/False?

True

False

Assuming the trends described in the previous question's figure are accurate (and hoping you got the axis labels right), which of the following are true? (Check all that apply.)

Increasing the training set size generally does not hurt an algorithm's performance, and it may help significantly.

Increasing the size of a neural network generally does not hurt an algorithm's performance, and it may help significantly.

Decreasing the training set size generally does not hurt an algorithm's performance, and it may help significantly.

Decreasing the size of a neural network generally does not hurt an algorithm's performance, and it may help significantly.

Deep Learning Notation

1 Neural Networks Notations.

General comments:

- superscript (i) will denote the i^{th} training example while superscript [l] will denote the l^{th} layer

Sizes:

· m : number of examples in the dataset

· n_x : input size

· n_y : output size (or number of classes)

· $n_h^{[l]}$: number of hidden units of the l^{th} layer

In a for loop, it is possible to denote $n_x = n_h^{[0]}$ and $n_y = n_h^{[\text{number of layers} + 1]}$.

· L : number of layers in the network.

Objects:

· $X \in \mathbb{R}^{n_x \times m}$ is the input matrix

· $x^{(i)} \in \mathbb{R}^{n_x}$ is the i^{th} example represented as a column vector

· $Y \in \mathbb{R}^{n_y \times m}$ is the label matrix

· $y^{(i)} \in \mathbb{R}^{n_y}$ is the output label for the i^{th} example

· $W^{[l]} \in \mathbb{R}^{\text{number of units in next layer} \times \text{number of units in the previous layer}}$ is the weight matrix, superscript [l] indicates the layer

· $b^{[l]} \in \mathbb{R}^{\text{number of units in next layer}}$ is the bias vector in the l^{th} layer

· $\hat{y} \in \mathbb{R}^{n_y}$ is the predicted output vector. It can also be denoted $a^{[L]}$ where L is the number of layers in the network.

Common forward propagation equation examples:

$a = g^{[l]}(W_x x^{(i)} + b_1) = g^{[l]}(z_1)$ where $g^{[l]}$ denotes the l^{th} layer activation function

$$\hat{y}^{(i)} = \text{softmax}(W_h h + b_2)$$

· General Activation Formula: $a_j^{[l]} = g^{[l]}(\sum_k w_{jk}^{[l]} a_k^{[l-1]} + b_j^{[l]}) = g^{[l]}(z_j^{[l]})$

· $J(x, W, b, y)$ or $J(\hat{y}, y)$ denote the cost function.

Examples of cost function:

· $J_{CE}(\hat{y}, y) = -\sum_{i=0}^m y^{(i)} \log \hat{y}^{(i)}$

· $J_1(\hat{y}, y) = \sum_{i=0}^m |y^{(i)} - \hat{y}^{(i)}|$

$(x, y) \quad x \in \mathbb{R}^{n_x}, y \in \{0, 1\}$
 m training examples: $\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(m)}, y^{(m)})\}$

$M = M_{\text{train}}$

$M_{\text{test}} = \# \text{test examples.}$

$$X = \begin{bmatrix} | & | & | & | \\ x^{(1)} & x^{(2)} & \dots & x^{(m)} \\ | & | & | & | \end{bmatrix} \begin{matrix} \uparrow \\ n_x \\ \downarrow \end{matrix}$$

$X \in \mathbb{R}^{n_x \times m}$ $X.\text{shape} = (n_x, m)$

$$Y = [y^{(1)} \ y^{(2)} \ \dots \ y^{(m)}]$$

$$Y \in \mathbb{R}^{1 \times m}$$

$$Y.\text{shape} = (1, m)$$

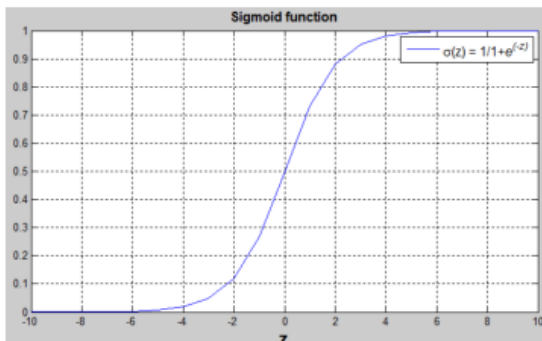
Logistic Regression

- **Binary classification:** discrete value output
- Logistic regression model:

$$\text{Given } x, \hat{y} = P(y = 1|x), \text{ where } 0 \leq \hat{y} \leq 1$$

The parameters used in Logistic regression are:

- The input features vector: $x \in \mathbb{R}^{n_x}$, where n_x is the number of features
- The training label: $y \in \{0, 1\}$
- The weights: $w \in \mathbb{R}^{n_x}$, where n_x is the number of features
- The threshold: $b \in \mathbb{R}$
- The output: $\hat{y} = \sigma(w^T x + b)$
- Sigmoid function: $s = \sigma(w^T x + b) = \sigma(z) = \frac{1}{1 + e^{-z}}$
- z large: $\sigma(z) \approx 1/1 + e^{-\infty} = 1/(1+0) = 1$
- z small: $\sigma(z) \approx 1/1 + e^{\infty} = 1/(\sigma(z) \approx 1/(1+\infty) = 0$
- Goal: learn parameters w and b so that $\hat{y}$ becomes a good estimate of the chance of $y=1$
- Easier to keep w and b as separate parameters



$$x_0 = 1, \quad x \in \mathbb{R}^{n_x + 1}$$

$$\hat{y} = \sigma(\Theta^T x)$$

$$\Theta = \begin{bmatrix} \Theta_0 \\ \Theta_1 \\ \Theta_2 \\ \vdots \\ \Theta_{n_x} \end{bmatrix} \begin{matrix} \} b \\ \} w \end{matrix}$$

What are the parameters of logistic regression?

W and b, both real numbers.

W and b, both n_x dimensional vectors.

W, an identity vector, and b, a real number.

W, an n_x dimensional vector, and b, a real number.

Logistic Regression Cost Function

$$\hat{y}^{(i)} = \sigma(w^T x^{(i)} + b), \text{ where } \sigma(z^{(i)}) = \frac{1}{1 + e^{-z^{(i)}}}$$

- Given $\{(x^{(1)}, y^{(1)}), \dots, (x^{(m)}, y^{(m)})\}$, we want $\hat{y}^{(i)} \approx y^{(i)}$ $x^{(i)}$ the i-th training example
- **Loss function** computes the error for a single training example (discrepancy between $\hat{y}^{(i)}$ and $y^{(i)}$)

$$L(\hat{y}^{(i)}, y^{(i)}) = \frac{1}{2}(\hat{y}^{(i)} - y^{(i)})^2$$

$$L(\hat{y}^{(i)}, y^{(i)}) = -(y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}))$$

If $y=1$: $L(\hat{y}, y) = -\log \hat{y} \leftarrow$ Want $\log \hat{y}$ large, want $\hat{y}$ large.
If $y=0$: $L(\hat{y}, y) = -\log(1 - \hat{y}) \leftarrow$ Want $\log(1 - \hat{y})$ large ... Want $\hat{y}$ small

- Minimizes squared error
- **Cost function** computes the error (average of the loss function) for the entire training set
 - Find the parameters w and b that minimize the overall cost function

$$J(w, b) = \frac{1}{m} \sum_{i=1}^m L(\hat{y}^{(i)}, y^{(i)}) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})]$$

What is the difference between the cost function and loss function for logistic regression?

The cost function computes the error for a single training example; the loss function is the average of the cost functions of the entire training set.

They are different names for the same function.

The loss function computes the error for a single training example; the cost function is the average of the loss functions of the entire training set.

- Justification of Function Choice

- For one training example:

Handwritten derivation showing the relationship between the probability $p(y|x)$ and the log-likelihood $\log p(y|x)$ for a single training example. It starts with two cases for $y=1$ and $y=0$, then combines them into a single expression for $p(y|x)$, and finally takes the logarithm to show it is the negative of the loss function $\mathcal{L}(\hat{y}, y)$.

$$\begin{aligned} &\left. \begin{aligned} \rightarrow \text{If } y=1: & p(y|x) = \hat{y} \\ \rightarrow \text{If } y=0: & p(y|x) = 1 - \hat{y} \end{aligned} \right\} p(y|x) \\ &\boxed{p(y|x) = \hat{y}^y (1-\hat{y})^{(1-y)}} \leftarrow \\ &\text{If } y=1: p(y|x) = \hat{y} \cdot \underbrace{(1-\hat{y})^0}_{=1} \\ &\text{If } y=0: p(y|x) = \hat{y}^0 \cdot (1-\hat{y})^{(1-0)} = 1 \times (1-\hat{y}) = 1-\hat{y} \\ &\uparrow \log p(y|x) = \log \hat{y}^y (1-\hat{y})^{(1-y)} = y \log \hat{y} + (1-y) \log (1-\hat{y}) \\ &= -\mathcal{L}(\hat{y}, y) \downarrow \end{aligned}$$

- Minimizing loss incentivizes/maximizes log probability

- For m training examples: (assumes iid training examples)

Handwritten derivation showing the log-likelihood function for m training examples. It starts with the product of individual probabilities, then takes the logarithm to convert the product into a sum. This sum is identified as the negative of the total loss, which is then scaled by $1/m$ to define the cost function $J(w, b)$.

$$\begin{aligned} \log p(\text{labels in training set}) &= \log \prod_{i=1}^m p(y^{(i)} | x^{(i)}) \\ \log p(\dots) &= \sum_{i=1}^m \log p(y^{(i)} | x^{(i)}) \\ &= \sum_{i=1}^m \underbrace{\log p(y^{(i)} | x^{(i)})}_{-\mathcal{L}(\hat{y}^{(i)}, y^{(i)})} \\ &= -\sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)}) \\ \text{Cost: } J(w, b) &= \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)}) \quad \text{Maximum likelihood estimator} \\ &\quad \text{(minimize)} \end{aligned}$$

1/m scaling factor for convenience

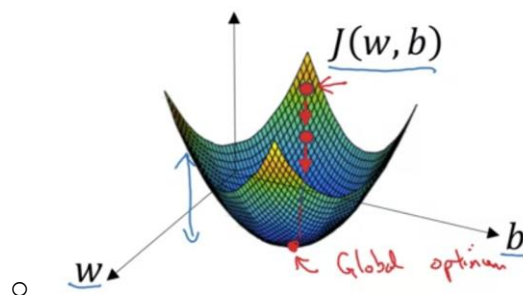
True or False: Minimizing the loss corresponds with maximizing $\log p(y|x)$.

False **True**

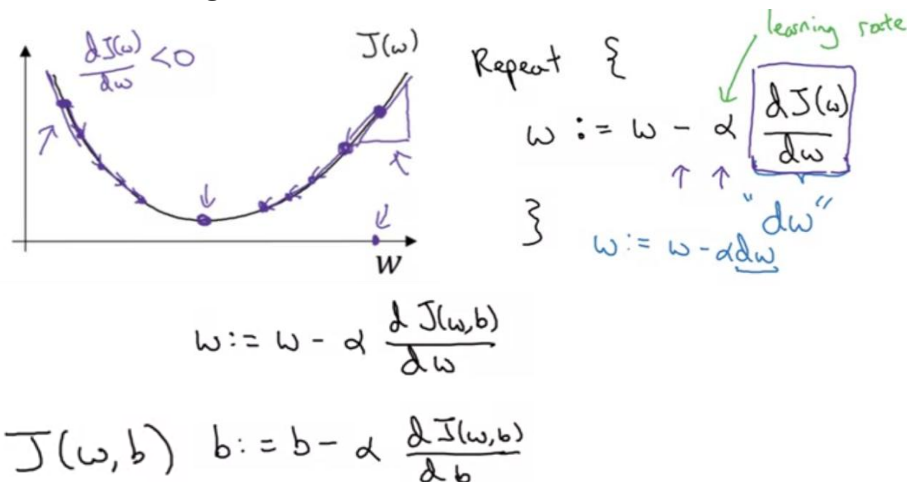
Gradient Descent

Recap: $\hat{y} = \sigma(w^T x + b)$, $\sigma(z) = \frac{1}{1+e^{-z}}$ ←

- $J(w, b) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)}) = -\frac{1}{m} \sum_{i=1}^m y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})$
- Train w and b on the training set. Find w, b that minimize the cost fn $J(w, b)$
- Logistic cost function is convex (one global minimum)



- Gradient descent takes tiny steps in the steepest direction
 - Learning rate α controls step size
 - Derivative is positive $\rightarrow$ subtract from w
 - Derivative is negative $\rightarrow$ add to w

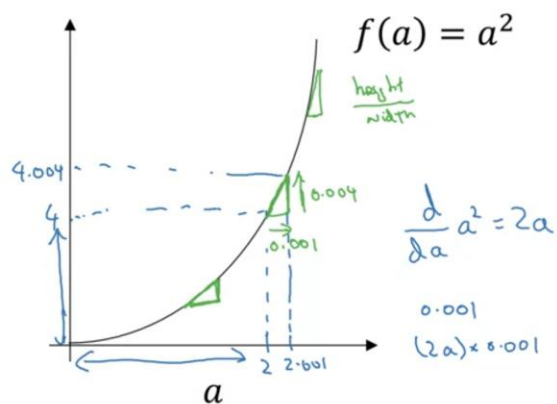


True or false: A convex function always has multiple local optima.

True **False** (logistic cost function has one, that's why we use it)

Derivatives (Basic Calculus)

- This is a layman walkthrough



$0.001 \leftarrow$
 $0.000000000001 \leftarrow$

$a=2$ $f(a)=4$
 $a=2.001$ $f(a) \approx 4.004$
 (4.004001)

slope (derivative) of $f(a)$ at
 $a=2$ is 4.

$\frac{d}{da} f(a) = 4$ when $a=2$.

$a=5$ $f(a)=25$
 $a=5.001$ $f(a) \approx 25.010$

$\frac{d}{da} f(a) = 10$ when $a=5$

$\frac{d}{da} f(a) = \frac{d}{da} a^2 = 2a$

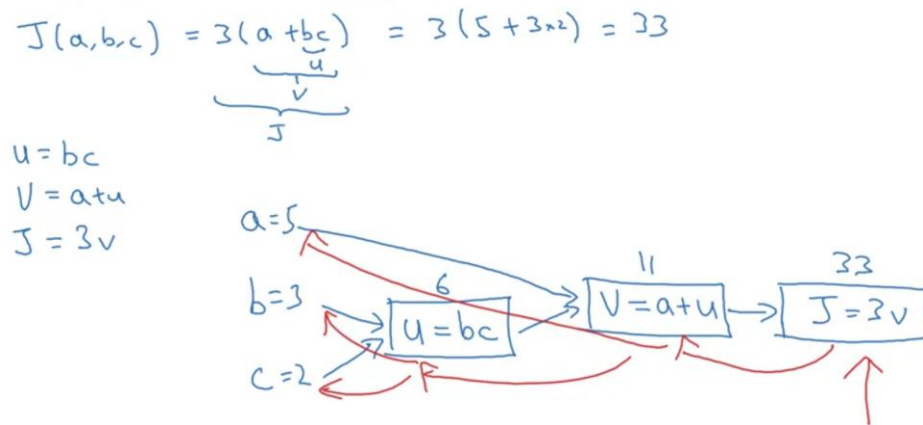
$f(a) = a^2$ $\frac{d}{da} f(a) = 2a$

$f(a) = a^3$ $\frac{d}{da} f(a) = 3a^2$
 $3 \times 2^2 = 12$

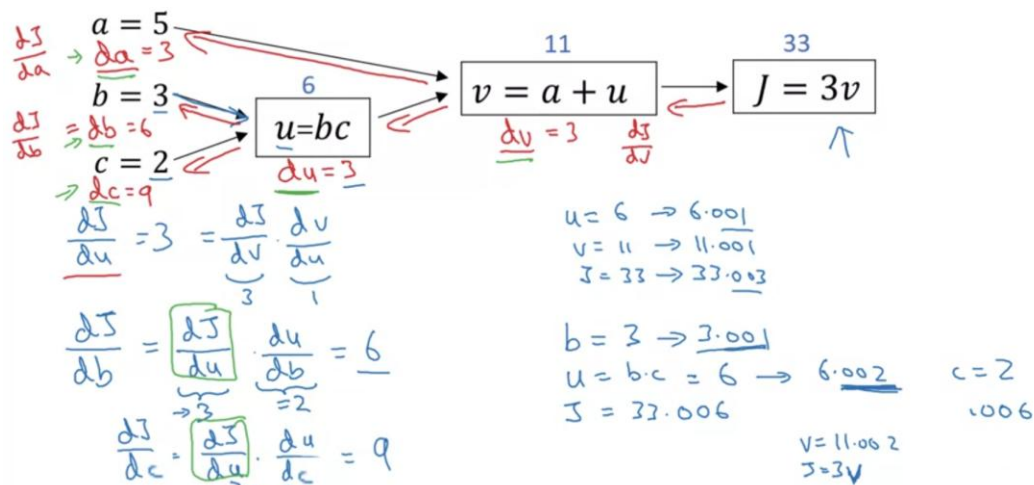
$f(a) = \log_e(a)$ $\frac{d}{da} f(a) = \frac{1}{a}$
 $\ln(a)$

Computation Graph

- Computational graph makes the forward blue arrows, the algorithm does the backwards propagation red arrows



- One step of **backward** propagation on a computation graph yields derivative of final output variable
- Calculating derivatives backwards through a computation graph



Uses chain rule

In this class, what does the coding convention `dvar` represent?

The derivative of a final output variable with respect to various intermediate quantities.

The derivative of input variables with respect to various intermediate quantities.

The derivative of any variable used in the code.

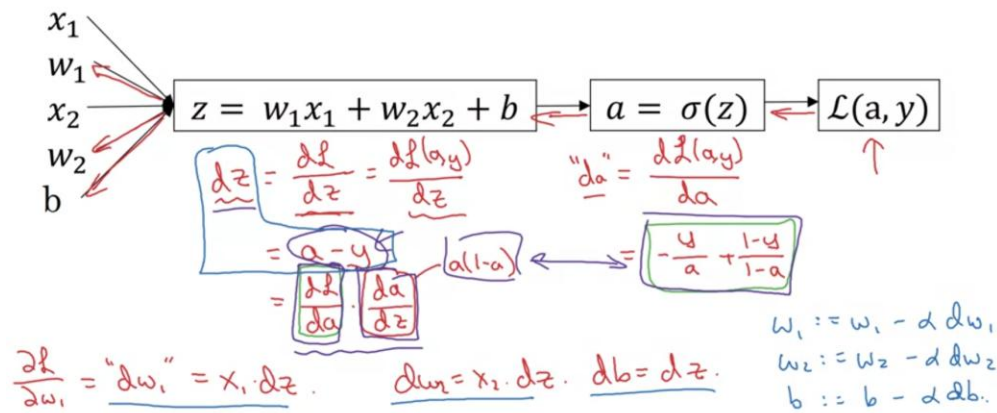
Logistic Regression Gradient Descent

- One training example

$$z = w^T x + b$$

$$\hat{y} = a = \sigma(z)$$

$$\mathcal{L}(a, y) = -(y \log(a) + (1 - y) \log(1 - a))$$



What is the simplified formula for the derivative of the loss with respect to z ?

$$a / (1-a)$$

$$a - y$$

$$a (1 - y)$$

- GD On m Training Examples

$$J(w, b) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(a^{(i)}, y^{(i)})$$

$$\rightarrow a^{(i)} = \hat{y}^{(i)} = \sigma(z^{(i)}) = \sigma(w^T x^{(i)} + b)$$

$$(x^{(i)}, y^{(i)})$$

$$\underline{dw_1^{(i)}}, \underline{dw_2^{(i)}}, \underline{db^{(i)}}$$

$$\frac{\partial}{\partial w_1} J(w, b) = \frac{1}{m} \sum_{i=1}^m \underbrace{\frac{\partial}{\partial w_1} \mathcal{L}(a^{(i)}, y^{(i)})}_{\frac{dw_1^{(i)}}{dw_1} - (x^{(i)}, y^{(i)})}$$

- Algorithm for each GD step

$$J=0; \underline{dw_1}=0; \underline{dw_2}=0; \underline{db}=0$$

For $i=1$ to m

$$z^{(i)} = w^T x^{(i)} + b$$

$$a^{(i)} = \sigma(z^{(i)})$$

$$J += -[y^{(i)} \log a^{(i)} + (1-y^{(i)}) \log(1-a^{(i)})]$$

$$\underline{dz^{(i)}} = a^{(i)} - y^{(i)}$$

$$dw_1 += x_1^{(i)} dz^{(i)}$$

$$dw_2 += x_2^{(i)} dz^{(i)}$$

$$db += dz^{(i)}$$

$\begin{matrix} \uparrow \\ dw_1 \\ \vdots \\ dw_n \end{matrix}$

$\begin{matrix} \downarrow \\ n=2 \end{matrix}$

$$J /= m$$

$$dw_1 /= m; dw_2 /= m; db /= m.$$

$$dw_1 = \frac{\partial J}{\partial w_1}$$

$$w_1 := w_1 - \alpha \underline{dw_1}$$

$$w_2 := w_2 - \alpha \underline{dw_2}$$

$$b := b - \alpha \underline{db}$$

Two for loop inefficiencies: over training m examples and over n features

In the for loop depicted in the video, why is there only one dw variable (i.e. no i superscripts in the for loop)?

Only one derivative is being computed.

The value of dw in the code is cumulative.

Only the derivative of one value is relevant.

Derivation of $dL/dz = a - y$

By the chain rule: $\frac{dL}{dz} = \frac{dL}{da} \times \frac{da}{dz}$

We'll do the following: 1. solve for $\frac{dL}{da}$, then

Step 1: $\frac{dL}{da}$

$$L = -(y \times \log(a) + (1 - y) \times \log(1 - a))$$

$$\frac{dL}{da} = -y \times \frac{1}{a} - (1 - y) \times \frac{1}{1-a} \times -1$$

We're taking the derivative with respect to a .

Remember that there is an additional -1 in the last term when we take the derivative of $(1 - a)$ with respect to a (remember the Chain Rule). Also note that the notational conventions are different in the ML world than the math world: here $\log$ always means the natural log.

$$\frac{dL}{da} = \frac{-y}{a} + \frac{1-y}{1-a}$$

We'll give both terms the same denominator:

$$\frac{dL}{da} = \frac{-y \times (1-a)}{a \times (1-a)} + \frac{a \times (1-y)}{a \times (1-a)}$$

Clean up the terms:

$$\frac{dL}{da} = \frac{-y + ay + a - ay}{a(1-a)}$$

So now we have:

$$\frac{dL}{da} = \frac{a-y}{a(1-a)}$$

Step 2: $\frac{da}{dz}$

$$\frac{da}{dz} = \frac{d}{dz} \sigma(z)$$

The derivative of a sigmoid has the form:

$$\frac{d}{dz} \sigma(z) = \sigma(z) \times (1 - \sigma(z))$$

You can look up why this derivation is of this form. For example, google “derivative of a sigmoid”, and you can see the derivation in detail.

Recall that $\sigma(z) = a$, because we defined “a”, the activation, as the output of the sigmoid activation function.

So we can substitute into the formula to get:

$$\frac{da}{dz} = a(1 - a)$$

Step 3: $\frac{dL}{dz}$

We’ll multiply step 1 and step 2 to get the result.

$$\frac{dL}{dz} = \frac{dL}{da} \times \frac{da}{dz}$$

$$\text{From step 1: } \frac{dL}{da} = \frac{a-y}{a(1-a)}$$

$$\text{From step 2: } \frac{da}{dz} = a(1 - a)$$

$$\frac{dL}{dz} = \frac{a-y}{a(1-a)} \times a(1 - a)$$

Notice that we can cancel factors to get this:

$$\frac{dL}{dz} = a - y$$

In Andrew’s notation, he’s referring to $\frac{dL}{dz}$ as dz .

So in the videos:

$$dz = a - y$$

Vectorization in Python

- Whenever possible, avoid for loops
- Dot product computes all at once vs looping

```
import numpy as np
z = np.dot(w, x) + b    [wTx = np * dot(w, x)]
```
- Scalable machine learning is done on GPU and CPU; they have SIMD (single data multiple data) parallel computing instructions. GPU is better but CPU works
- Other numpy functions: zeroes (empty matrix of args dimensions), exp, log, abs, maximum

Vectorizing Logistic Regression Predictions

$J = 0, \text{ ~~dw1 = 0, dw2 = 0~~ }, db = 0$ $dw = np.zeros((n_x, 1))$
 $\rightarrow \text{for } i = 1 \text{ to } m:$
 $z^{(i)} = w^T x^{(i)} + b$
 $a^{(i)} = \sigma(z^{(i)})$
 $J += -[y^{(i)} \log a^{(i)} + (1 - y^{(i)}) \log(1 - a^{(i)})]$
 $\frac{dz^{(i)}}{da^{(i)}} = a^{(i)} - y^{(i)}$
 $\frac{dw_1}{dz^{(i)}} = x_1^{(i)} \frac{dz^{(i)}}{da^{(i)}} \quad n_x = 2$ $dw += x^{(i)} dz^{(i)}$
 $\frac{dw_2}{dz^{(i)}} = x_2^{(i)} \frac{dz^{(i)}}{da^{(i)}}$
 $db += dz^{(i)}$
 $J = J/m, \text{ ~~dw1 = dw1/m, dw2 = dw2/m~~ }, db = db/m$
 $dw /= m.$

$\rightarrow \begin{cases} z^{(1)} = w^T x^{(1)} + b \\ a^{(1)} = \sigma(z^{(1)}) \end{cases} \quad \begin{cases} z^{(2)} = w^T x^{(2)} + b \\ a^{(2)} = \sigma(z^{(2)}) \end{cases} \quad \begin{cases} z^{(3)} = w^T x^{(3)} + b \\ a^{(3)} = \sigma(z^{(3)}) \end{cases}$
 $X = \begin{bmatrix} x^{(1)} & x^{(2)} & \dots & x^{(m)} \end{bmatrix} \quad \begin{matrix} (n_x, m) \\ \mathbb{R}^{n_x \times m} \end{matrix}$
 $\bar{z} = \begin{bmatrix} z^{(1)} & z^{(2)} & \dots & z^{(m)} \end{bmatrix} = w^T X + \begin{bmatrix} b & b & \dots & b \end{bmatrix} = \begin{bmatrix} w^T x^{(1)} + b & w^T x^{(2)} + b & \dots & w^T x^{(m)} + b \end{bmatrix}$
 $\rightarrow \bar{z} = np.dot(w.T, X) + b$ $\bar{z} (1, m) \quad \mathbb{R}$ "Broadcasting"
 $A = \begin{bmatrix} a^{(1)} & a^{(2)} & \dots & a^{(m)} \end{bmatrix} = \sigma(\bar{z})$

What are the dimensions of matrix X?

(m,) (m, 1) **(n_x, m)** (m, n_x)

Vectorizing Logistic Regression Gradient Computation

$$\begin{aligned}
 dz^{(i)} &= a^{(i)} - y^{(i)} & dz^{(i)} &= a^{(i)} - y^{(i)} & \dots \\
 dZ &= [dz^{(1)} \ dz^{(2)} \ \dots \ dz^{(m)}] \quad \leftarrow \\
 A &= [a^{(1)} \ \dots \ a^{(m)}] & Y &= [y^{(1)} \ \dots \ y^{(m)}] \\
 \rightarrow dZ &= A - Y = [a^{(1)} - y^{(1)} \ \dots \ a^{(m)} - y^{(m)}]
 \end{aligned}$$

$$\begin{aligned}
 \left[\begin{array}{l} dw = 0 \\ dw += \frac{x^{(1)} dz^{(1)}}{m} \\ dw += \frac{x^{(2)} dz^{(2)}}{m} \\ \vdots \\ dw /= m \end{array} \right] & \quad \left[\begin{array}{l} db = 0 \\ db += dz^{(1)} \\ db += dz^{(2)} \\ \vdots \\ db += dz^{(m)} \\ db /= m \end{array} \right]
 \end{aligned}$$

$$\begin{aligned}
 db &= \frac{1}{m} \sum_{i=1}^m dz^{(i)} \\
 &= \frac{1}{m} \text{np.sum}(dZ) \\
 dw &= \frac{1}{m} X dZ^T \\
 &= \frac{1}{m} \begin{bmatrix} x^{(1)} & \dots & x^{(m)} \\ 1 & & 1 \end{bmatrix} \begin{bmatrix} dz^{(1)} \\ \vdots \\ dz^{(m)} \end{bmatrix} \\
 &= \frac{1}{m} [x^{(1)} dz^{(1)} + \dots + x^{(m)} dz^{(m)}] \quad n \times 1
 \end{aligned}$$

Implementing Logistic Regression Vectorization

$$\begin{aligned}
 J &= 0, \quad dw_1 = 0, \quad dw_2 = 0, \quad db = 0 \\
 \text{for } i &= 1 \text{ to } m: \\
 z^{(i)} &= w^T x^{(i)} + b \quad \leftarrow \\
 a^{(i)} &= \sigma(z^{(i)}) \quad \leftarrow \\
 J &+= -[y^{(i)} \log a^{(i)} + (1 - y^{(i)}) \log(1 - a^{(i)})] \\
 dz^{(i)} &= a^{(i)} - y^{(i)} \quad \leftarrow \\
 \left[\begin{array}{l} dw_1 += x_1^{(i)} dz^{(i)} \\ dw_2 += x_2^{(i)} dz^{(i)} \end{array} \right] & \quad dw += x^{(i)} * dz^{(i)} \\
 db &+= dz^{(i)} \\
 J &= J/m, \quad dw_1 = dw_1/m, \quad dw_2 = dw_2/m \\
 db &= db/m
 \end{aligned}$$

$$\begin{aligned}
 \text{for } \text{iter} &\text{ in range}(1000): \quad \leftarrow \\
 Z &= w^T X + b \\
 &= \text{np.dot}(w.T, X) + b \\
 A &= \sigma(Z) \\
 dZ &= A - Y \\
 dw &= \frac{1}{m} X dZ^T \\
 db &= \frac{1}{m} \text{np.sum}(dZ) \\
 w &:= w - \alpha dw \\
 b &:= b - \alpha db
 \end{aligned}$$

- Still need a for loop for iterations, there's no getting around that. But everything within one gradient descent iteration can be vectorized and efficient

How do you compute the derivative of b in one line of code in Python numpy?

$$\begin{aligned}
 1 * m(\text{np.sum}(dZ)) & \quad m(\text{np.sum}(dZ)) \\
 1 / m * (\text{np.sum}(dZ)) & \quad 1 - m(\text{np.sum}(dZ))
 \end{aligned}$$

Broadcasting in Python

- Makes Python code faster
- Example:

Calories from Carbs, Proteins, Fats in 100g of different foods:

	Apples	Beef	Eggs	Potatoes
Carb	56.0	0.0	4.4	68.0
Protein	1.2	104.0	52.0	8.0
Fat	1.8	135.0	99.0	0.9

$= A$
(3,4)

59 cal
 $\frac{56}{59} \approx 94.9\%$

Calculate % of calories from Carb, Protein, Fat. Can you do this without explicit for-loop?

`cal = A.sum(axis=0)`

calculates vertical sums (total cal)

`percentage = 100*A/cal.reshape(1,4)`

gives percentage for each cell

reshape is redundant here but it's good practice for when the shape is unknown

- Examples:

$$\begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix} + \begin{bmatrix} 100 \\ 100 \\ 100 \\ 100 \end{bmatrix} 100 = \begin{bmatrix} 101 \\ 102 \\ 103 \\ 104 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} + \begin{bmatrix} 100 & 200 & 300 \\ 100 & 200 & 300 \end{bmatrix} = \begin{bmatrix} 101 & 202 & 303 \\ 104 & 205 & 306 \end{bmatrix}$$

$(m,n) \quad (2,3) \quad (1,n) \rightsquigarrow (m,n) \quad (2,3)$

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} + \begin{bmatrix} 100 & 100 & 100 \\ 200 & 200 & 200 \end{bmatrix} = \begin{bmatrix} 101 & 102 & 103 \\ 204 & 205 & 206 \end{bmatrix}$$

$(m,n) \quad (m,1) \downarrow (m,n)$

- General principle: (m, n) matrix {operation} $(1, n) \rightarrow$ [copy n times into] (m, n)
 (m, n) matrix {operation} $(m, 1) \rightarrow$ [copy m times into] (m, n)
 $(m, 1)$ matrix {operation} real # $\rightarrow$ [copy m times]

Which of the following numpy line of code would sum the values in a matrix A vertically?

A.sum(axis = 0)

A.sum(axis = 1)

A.sum(axis)

Numpy Vectors

- Numpy won't throw an error and stop if there is a bug
- Example: **rank 1 arrays**

```
In [1]: import numpy as np
        a = np.random.randn(5)

In [2]: print(a)
[ 0.50290632 -0.29691149  0.95429604 -0.82126861 -1.46269164]

In [3]: print(a.shape)
(5,)

In [4]: print(a.T)
[ 0.50290632 -0.29691149  0.95429604 -0.82126861 -1.46269164]

In [5]: print(np.dot(a,a.T))
4.06570109321

In [6]: a = np.random.randn(5,1)
        print(a)
[[-0.0967311 ]
 [-2.38617377]
 [-0.3243588 ]
 [-0.96216349]
 [ 0.54410384]]

In [7]: print(a.T)
[[-0.0967311  -2.38617377 -0.3243588  -0.96216349  0.54410384]]

In [8]: print(np.dot(a,a.T))
[[ 0.00935691  0.23081721  0.03137558  0.09307113 -0.05263176]
 [ 0.23081721  5.69382526  0.77397645  2.29588928 -1.2983263 ]
 [ 0.03137558  0.77397645  0.10520863  0.31208619 -0.17648487]
 [ 0.09307113  2.29588928  0.31208619  0.92575858 -0.52351684]
 [-0.05263176 -1.2983263  -0.17648487 -0.52351684  0.29604898]]
```

The dot product of a and a^T should be an $n \times n$ matrix!

$a = \text{np.random.randn}(5) \rightarrow a.\text{shape}(5,)$

[rank 1 array] behaves neither as a row or column vector (don't use)

$a = \text{np.random.randn}(5, 1) \rightarrow a.\text{shape}(5, 1) \rightarrow$ column vector

$a = \text{np.random.randn}(1, 5) \rightarrow a.\text{shape}(1, 5) \rightarrow$ row vector

$\text{assert}(a.\text{shape} == (5,1))$ make sure of the dimensions, good documentation

$a = a.\text{shape}((5,1))$ reshape a rank 1 array

What kind of array has dimensions in this format: (10,)?

A rank 1 array

A rank 0 array

An identity array

What does a neuron compute?

A neuron computes a linear function $z = Wx + b$ followed by an activation function

A neuron computes a function g that scales the input x linearly ($Wx + b$)

A neuron computes an activation function followed by a linear function $z = Wx + b$

A neuron computes the mean of all features before applying the output to an activation function

Suppose that $\hat{y} = 0.5$ and $y = 0$. What is the value of the "logistic loss"?

$$\mathcal{L}(\hat{y}, y) = -(y \log \hat{y} + (1 - y) \log(1 - \hat{y}))$$

0.5

$+\infty$

0.693 $-[0 \cdot \ln(0.5) + (1-0) \log(1-0.5)] = -[0 + \ln(0.50)]$

Suppose x is a (8, 1) array. Which of the following is a valid reshape?

`x.reshape(-1, 3)`

`x.reshape(2, 4, 4)`

`x.reshape(1, 4, 3)`

`x.reshape(2, 2, 2)`

Consider the following random arrays a , b , and c :

$a = np.random.randn(3,4)$ # $a.shape = (3,4)$

$b = np.random.randn(1,4)$ # $b.shape = (1,4)$

$c = a + b$

What will be the shape of c ?

The computation cannot happen because it is not possible to broadcast more than one dimension.

$c.shape = (3, 4)$

$c.shape = (1, 4)$

$c.shape = (3, 1)$

Consider the following random arrays a and b :

$a = np.random.randn(4,3)$ # $a.shape = (4,3)$

$b = np.random.randn(1,3)$ # $b.shape = (1,3)$

$c = a * b$

What will be the shape of c ?

$c.shape = (4, 3)$

$c.shape = (1, 3)$

The computation cannot happen because it is not possible to broadcast more than one dimension.

The computation cannot happen because the sizes don't match.

Suppose our input batch consists of 8 grayscale images, each of dimension 8x8. We reshape these images into feature column vectors x^j . Remember that $X = [x^{(1)} x^{(2)} \dots x^{(8)}]$. What is the dimension of X ?

(64, 8)

(8, 8, 8)

(512, 1)

(8, 64)

Recall that `np.dot(a, b)` performs a matrix multiplication on `a` and `b`, whereas `a * b` performs an element-wise multiplication.

Consider the two following random arrays `a` and `b`:

```
a = np.random.randn(12288, 150) # a.shape = (12288, 150)
```

```
b = np.random.randn(150, 45) # b.shape = (150, 45)
```

```
c = np.dot(a, b)
```

What is the shape of `c`?

```
c.shape = (150, 150)
```

```
c.shape = (12288, 150)
```

The computation cannot happen because the sizes don't match. It's going to be "Error"!

```
c.shape = (12288, 45)
```

Consider the following code snippet:

```
a.shape = (3,4)
```

```
b.shape = (4,1)
```

```
for i in range(3):
```

```
for j in range(4):
```

```
c[i][j] = a[i][j] + b[j]
```

How do you vectorize this?

```
c = a.T + b
```

```
c = a + b
```

```
c = a.T + b.T
```

```
c = a + b.T
```


Consider the code snippet:

a.shape = (3,3)

b.shape = (3,3)

c = *a* ** 2 + *b.T* ** 2

Which of the following gives an equivalent output for *c*?

for i in range(3):

for j in range(3):

c[i][j] = *a*[i][j]**2 + *b*[i][j]**2

The computation cannot happen because the sizes don't match. It's going to be an "Error"!

for i in range(3):

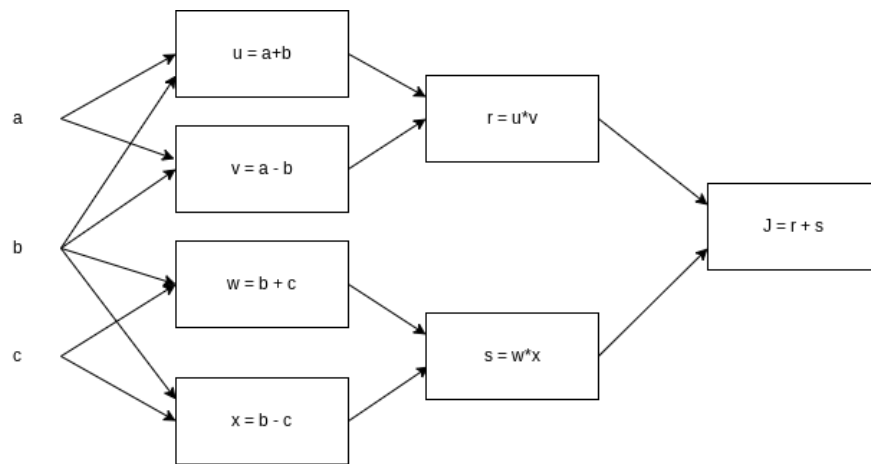
c[i] = *a*[i]**2 + *b*[i]**2

for i in range(3):

for j in range(3):

***c*[i][j] = *a*[i][j]**2 + *b*[j][i]**2**

Consider the following computational graph.



What is the output of J ?

$$a^2 + b^2 - c^2$$

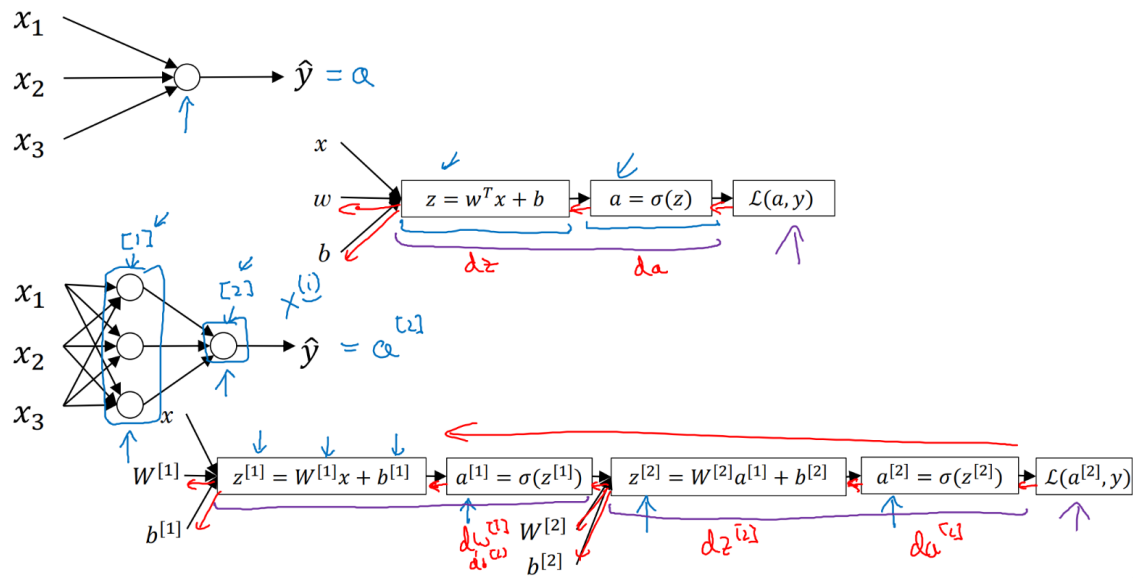
$$a^2 - c^2$$

$$(a - b) * (a - c)$$

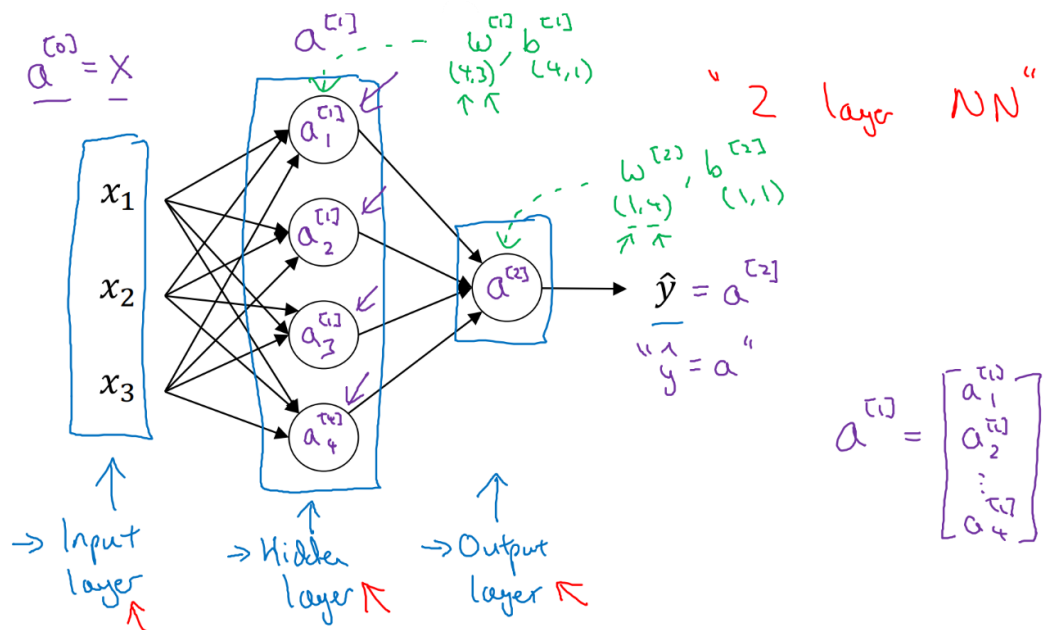
$$\mathbf{a^2 - b^2} \quad J = r + s = u * v + w * x = (a+b)(a-b) + (b+c)(b-c) = a^2 - b^2 + b^2 - c^2 = a^2 - c^2$$

Shallow Neural Networks

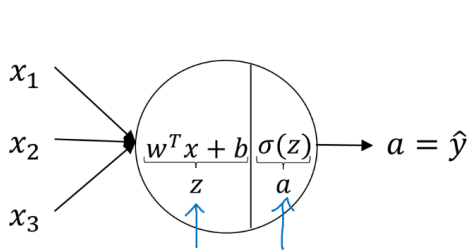
- Repeat a logistic regression twice



- Representation

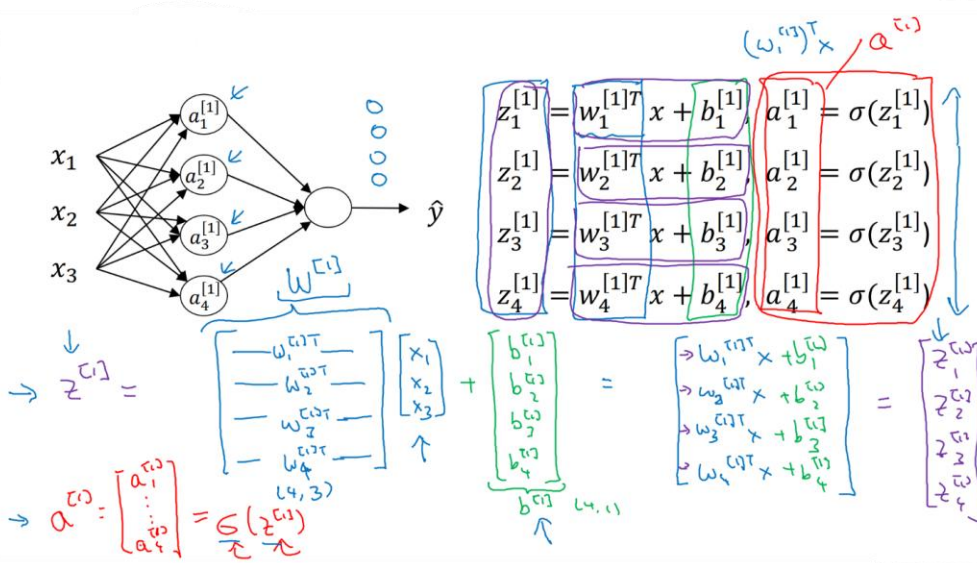
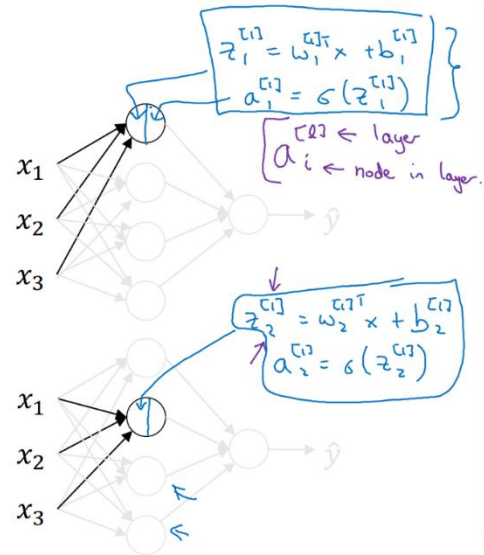
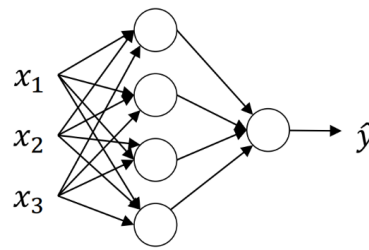


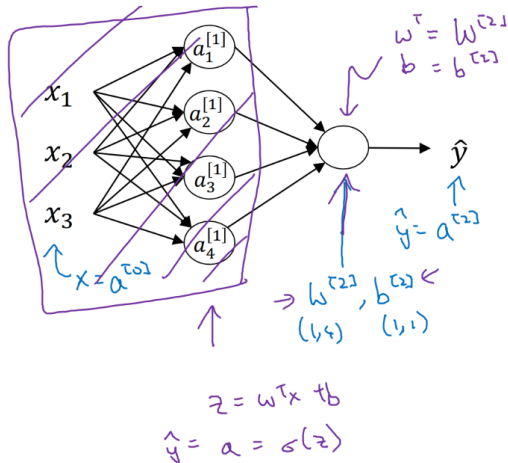
- Computing NN Output



$$z = w^T x + b$$

$$a = \sigma(z)$$



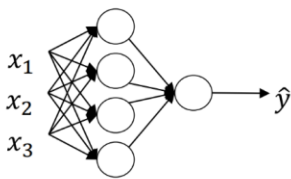


Given input x :

$$\begin{aligned} \rightarrow z^{[1]} &= W^{[1]} a^{[0]} + b^{[1]} \\ &\quad \begin{matrix} (4,1) & (4,3) & (3,1) & (4,1) \end{matrix} \\ \rightarrow a^{[1]} &= \sigma(z^{[1]}) \\ &\quad \begin{matrix} (4,1) \end{matrix} \\ \rightarrow z^{[2]} &= W^{[2]} a^{[1]} + b^{[2]} \\ &\quad \begin{matrix} (1,1) & (1,4) & (4,1) & (1,1) \end{matrix} \\ \rightarrow a^{[2]} &= \sigma(z^{[2]}) \\ &\quad \begin{matrix} (1,1) \end{matrix} \end{aligned}$$

Vectorized implementation of computing the output of these four logistic regression units in the hidden layer and one logistic regression unit in the output layer

Vectorizing Across Multiple Examples



$$\begin{aligned} x &\rightarrow a^{[1]} = \hat{y} \\ x^{(1)} &\rightarrow a^{1} = \hat{y}^{(1)} \\ x^{(2)} &\rightarrow a^{[1](2)} = \hat{y}^{(2)} \\ &\vdots \\ x^{(m)} &\rightarrow a^{[1](m)} = \hat{y}^{(m)} \end{aligned}$$

$a^{[1](i)}$ ← example i layer 2

$$\begin{aligned} \left\{ \begin{aligned} z^{[1]} &= W^{[1]} x + b^{[1]} \\ a^{[1]} &= \sigma(z^{[1]}) \\ z^{[2]} &= W^{[2]} a^{[1]} + b^{[2]} \\ a^{[2]} &= \sigma(z^{[2]}) \end{aligned} \right\} \\ \rightarrow \text{for } i = 1 \text{ to } m, \\ &\quad z^{[1](i)} = W^{[1]} x^{(i)} + b^{[1]} \\ &\quad a^{[1](i)} = \sigma(z^{[1](i)}) \\ &\quad z^{[2](i)} = W^{[2]} a^{[1](i)} + b^{[2]} \\ &\quad a^{[2](i)} = \sigma(z^{[2](i)}) \end{aligned}$$

for $i = 1$ to m :

$$\begin{aligned} z^{[1](i)} &= W^{[1]} x^{(i)} + b^{[1]} \\ a^{[1](i)} &= \sigma(z^{[1](i)}) \\ z^{[2](i)} &= W^{[2]} a^{[1](i)} + b^{[2]} \\ a^{[2](i)} &= \sigma(z^{[2](i)}) \end{aligned}$$

$$X = \begin{bmatrix} x_1 & x_2 & \dots & x_n \\ \vdots & \vdots & & \vdots \\ x_m & x_m & \dots & x_n \end{bmatrix}$$

↑ $(n \times m)$ ↑

↑ m examples ↑

↑ hidden units ↑

$$\begin{aligned} z^{[1]} &= W^{[1]} X + b^{[1]} \\ \rightarrow A^{[1]} &= \sigma(z^{[1]}) \\ \rightarrow z^{[2]} &= W^{[2]} A^{[1]} + b^{[2]} \\ \rightarrow A^{[2]} &= \sigma(z^{[2]}) \end{aligned}$$

$$z^{[1]} = \begin{bmatrix} z^{1} & z^{[1](2)} & \dots & z^{[1](m)} \\ 1 & 1 & \dots & 1 \end{bmatrix}$$

$$A^{[1]} = \begin{bmatrix} a^{1} & a^{[1](2)} & \dots & a^{[1](m)} \\ 1 & 1 & \dots & 1 \end{bmatrix}$$

↑ m examples ↑

↑ hidden units ↑

Justification:

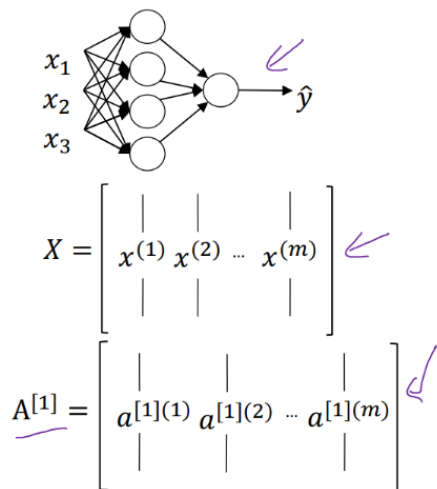
$$z^{[1]}(i) = w^{[1]} x^{(i)} + b^{[1]}, \quad z^{[1]}(2) = w^{[1]} x^{(2)} + b^{[1]}, \quad z^{[1]}(3) = w^{[1]} x^{(3)} + b^{[1]}$$

$$w^{[1]} = \begin{bmatrix} \vdots \\ \vdots \\ \vdots \end{bmatrix} \quad w^{[1]} x^{(1)} = \begin{bmatrix} \vdots \\ \vdots \\ \vdots \end{bmatrix} \quad w^{[1]} x^{(2)} = \begin{bmatrix} \vdots \\ \vdots \\ \vdots \end{bmatrix} \quad w^{[1]} x^{(3)} = \begin{bmatrix} \vdots \\ \vdots \\ \vdots \end{bmatrix}$$

$$w^{[1]} \begin{bmatrix} | & | & | & \dots \\ x^{(1)} & x^{(2)} & x^{(3)} & \dots \\ | & | & | & \dots \end{bmatrix} = \begin{bmatrix} \vdots & \vdots & \vdots & \dots \\ \vdots & \vdots & \vdots & \dots \\ \vdots & \vdots & \vdots & \dots \end{bmatrix} = \begin{bmatrix} z^{[1]}(1) & z^{[1]}(2) & z^{[1]}(3) & \dots \\ | & | & | & \dots \\ | & | & | & \dots \end{bmatrix} = z^{[1]}$$

$z^{[1]} = w^{[1]} X + b^{[1]}$

$w^{[1]} x^{(i)} = z^{[1]}(i)$



for $i = 1$ to m

- $z^{[1]}(i) = w^{[1]} x^{(i)} + b^{[1]}$
- $a^{[1]}(i) = \sigma(z^{[1]}(i))$
- $z^{[2]}(i) = w^{[2]} a^{[1]}(i) + b^{[2]}$
- $a^{[2]}(i) = \sigma(z^{[2]}(i))$

$x = a^{[0]}, \quad x^{(i)} = a^{[0]}(i)$

$Z^{[1]} = W^{[1]} X + b^{[1]} \leftarrow w^{[1]} A^{[0]} + b^{[1]}$

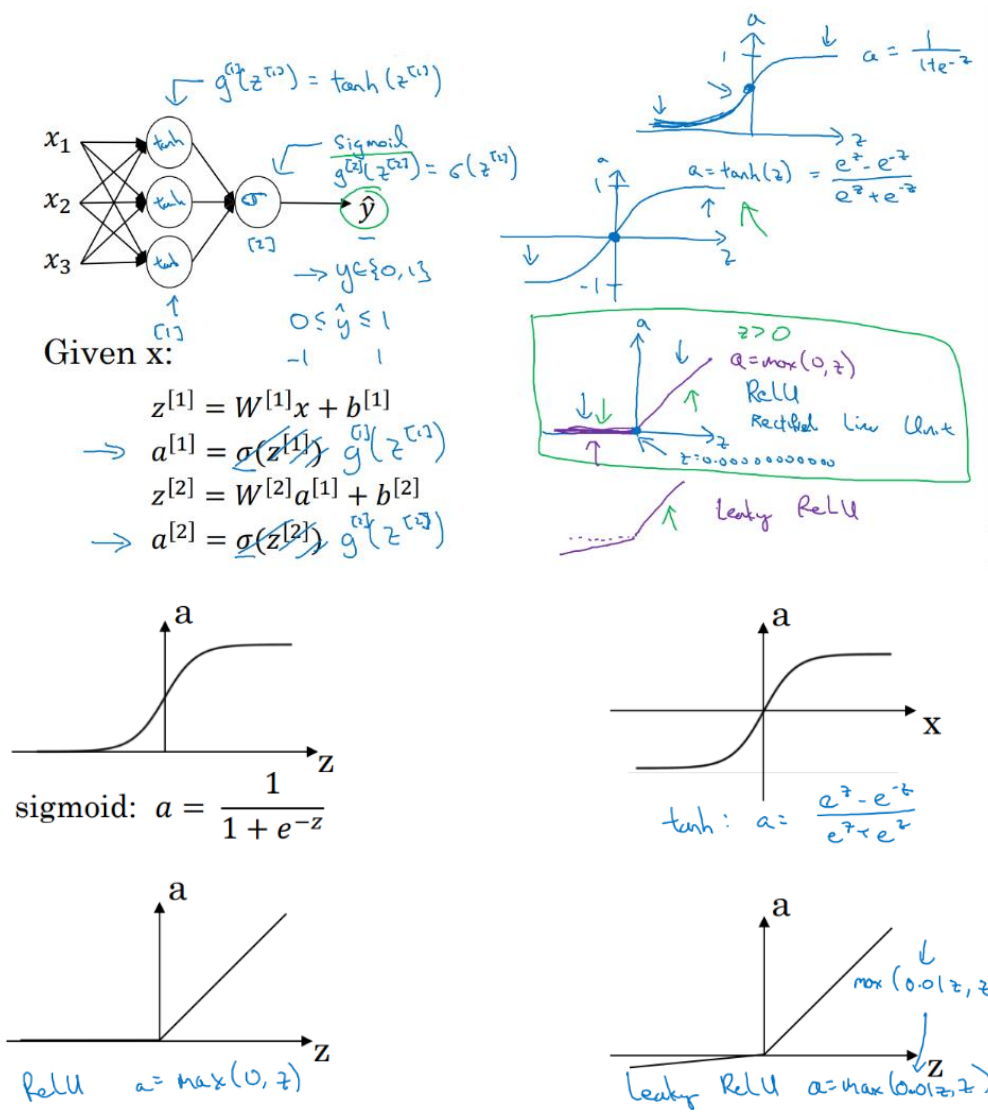
$A^{[1]} = \sigma(Z^{[1]})$

$Z^{[2]} = W^{[2]} A^{[1]} + b^{[2]}$

$A^{[2]} = \sigma(Z^{[2]})$

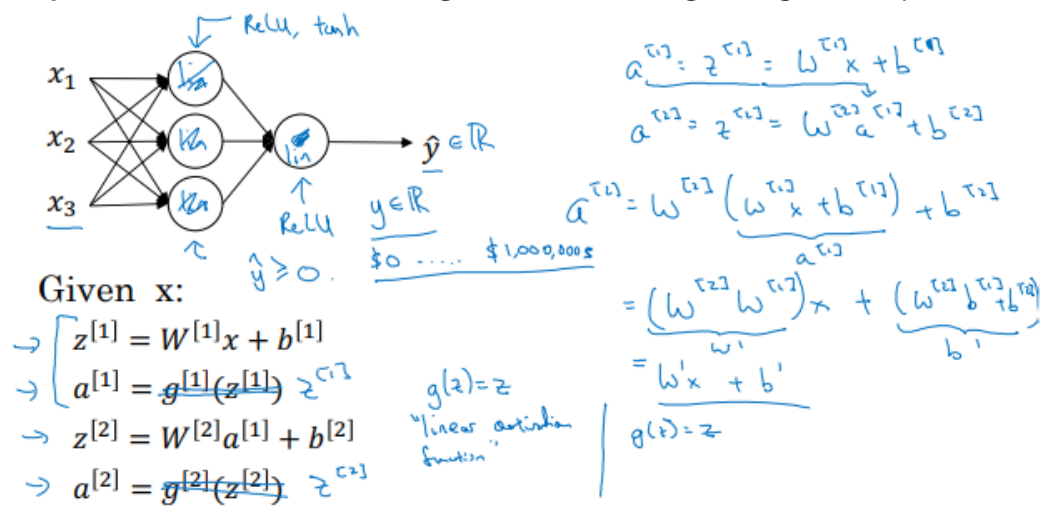
Activation Functions

- **Sigmoid** isn't always the best option. Use only for binary classification, since 0 and 1 make sense
- **Tanh** (hyperbolic tangent) $a = (e^z - e^{-z}) / (e^z + e^{-z})$. Almost always works better than sigmoid. Shifted version of sigmoid function [intersects (0,0) and rescaled to go from -1 to 1]. "Centers" data, makes learning easier for the next layer
 - Downside of both: large or small $z \rightarrow$ derivatives becomes small, which slows down gradient descent
- **ReLU** (rectified linear unit): $a = \max(0, z)$. Derivative is 1 if z is positive, 0 if z is negative
 - Limitation: the derivative is zero when z is negative
- **Leaky ReLU**: $a = \max(0.01z, z)$. Like ReLU but has a slight slope to the left of $z = 0$



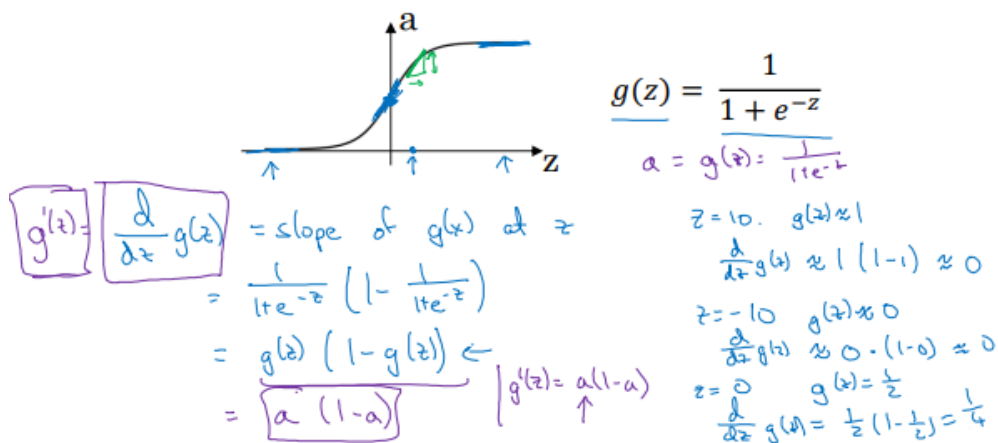
Purpose of Non-Linear Activation Functions

- Linear/identity activation functions → NN just outputs a linear function of the input. No matter how many layers, it's only computing a linear activation function, so you may as well not have any hidden layers
- All linear hidden layers and sigmoid output layer is the same as logistic regression without a hidden layer
- The composition of two linear functions is itself a linear function, so unless you throw non-linearity in there you're not computing more interesting functions as you go deeper the network
- Only use linear activation if using machine learning for regression problems



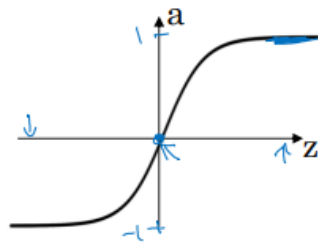
Derivatives of Activation Functions

- Sigmoid
 - $g'(z) = a(1-a)$



- Tanh

- $g'(z) = 1 - a^2$



$$g(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

$$g'(z) = \frac{d}{dz} g(z) = \text{slope of } g(z) \text{ at } z$$

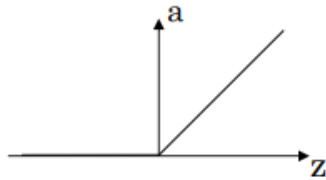
$$= 1 - \tanh(z)^2 \leftarrow$$

$$a = g(z), \quad g'(z) = 1 - a^2$$

$$\begin{aligned} z=10 & \quad \tanh(z) \approx 1 \\ & \quad g'(z) \approx 0 \\ z=-10 & \quad \tanh(z) \approx -1 \\ & \quad g'(z) \approx 0 \\ z=0 & \quad \tanh(z) = 0 \\ & \quad g'(z) = 1 \end{aligned}$$

- ReLU and Leaky ReLU

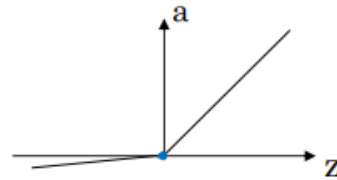
- ReLU: $g'(z) = 0$ if $z < 0$, 1 if $z \geq 0$
- Leaky ReLU: $g'(z) = 0.01$ if $z < 0$, 1 if $z \geq 0$



ReLU

$$g(z) = \max(0, z)$$

$$\rightarrow g'(z) = \begin{cases} 0 & \text{if } z < 0 \\ 1 & \text{if } z \geq 0 \end{cases}$$



Leaky ReLU

$$g(z) = \max(0.01z, z)$$

$$g'(z) = \begin{cases} 0.01 & \text{if } z < 0 \\ 1 & \text{if } z \geq 0 \end{cases}$$

Implementing Gradient Descent for Neural Networks

- For binary classification:

Parameters: $W^{[1]}, b^{[1]}, W^{[2]}, b^{[2]}$
 $(n^{[1]}, n^{[2]})$ $(n^{[2]}, 1)$ $(n^{[1]}, 1)$ $(n^{[2]}, 1)$ $n_x = n^{[0]}, n^{[1]}, n^{[2]} = 1$

Cost function: $J(W^{[1]}, b^{[1]}, W^{[2]}, b^{[2]}) = \frac{1}{m} \sum_{i=1}^m \ell(\hat{y}_i, y_i)$
 $\uparrow \quad \uparrow \quad \uparrow$ $a^{[2]}$

Gradient descent:

→ Repeat {
 → Compute predictions $(\hat{y}^{(i)}, i=1, \dots, m)$
 $\frac{dJ}{dW^{[1]}} = \frac{\partial J}{\partial W^{[1]}}$, $\frac{db^{[1]}}{dJ} = \frac{\partial J}{\partial b^{[1]}}$, ...
 $W^{[1]} := W^{[1]} - \alpha \frac{dJ}{dW^{[1]}}$
 $b^{[1]} := b^{[1]} - \alpha \frac{db^{[1]}}{dJ}$
 $W^{[2]} := \dots$ $b^{[2]} := \dots$

Forward propagation:

$$z^{[1]} = W^{[1]}x + b^{[1]}$$

$$A^{[1]} = g^{[1]}(z^{[1]}) \leftarrow$$

$$z^{[2]} = W^{[2]}A^{[1]} + b^{[2]}$$

$$A^{[2]} = g^{[2]}(z^{[2]}) = \sigma(z^{[2]})$$

Back propagation:

$$dz^{[2]} = A^{[2]} - Y \leftarrow Y = [y^{(1)} y^{(2)} \dots y^{(m)}]$$

$$dW^{[2]} = \frac{1}{m} dz^{[2]} A^{[1]T}$$

$$db^{[2]} = \frac{1}{m} \text{np.sum}(dz^{[2]}, \text{axis}=1, \text{keepdims}=\text{True})$$

$$dz^{[1]} = \underbrace{W^{[2]T}}_{(n^{[2]}, m)} dz^{[2]} \times \underbrace{g^{[2]'}(z^{[1]})}_{\text{element-wise product}} \quad (n^{[1]}, m)$$

$$dW^{[1]} = \frac{1}{m} dz^{[1]} x^T$$

$$db^{[1]} = \frac{1}{m} \text{np.sum}(dz^{[1]}, \text{axis}=1, \text{keepdims}=\text{True})$$

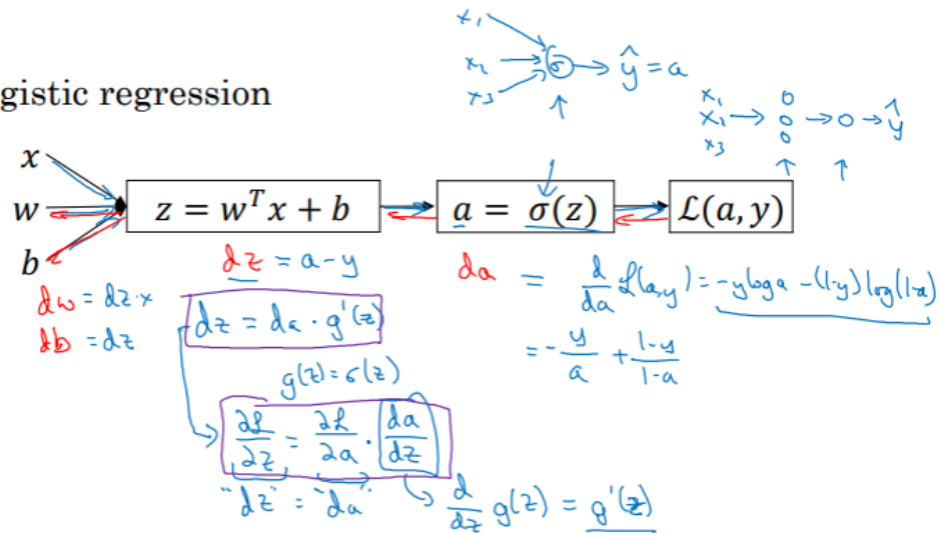
$(n^{[1]}, 1)$ $(n^{[1]}, 1)$ reshape ↑

- Keepdims= true ensures Python outputs an n2 x 1 vector, not a rank 1 array
- 4 equations for forward propagation, 6 equations for backward propagation

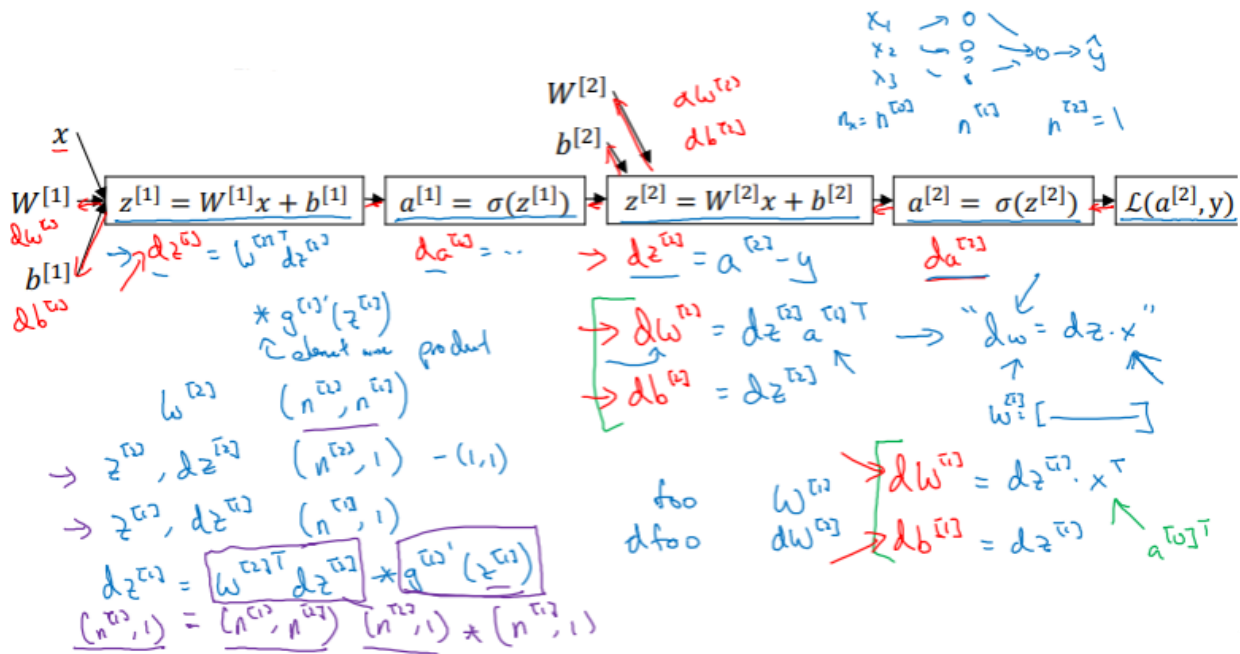
Backpropagation Derivation

- Computing gradients
 - Uses the chain rule

Logistic regression



- Neural network gradients
 - w and dw always have the same dimensions



- Summary of gradient descent

$$dz^{[2]} = a^{[2]} - y$$

$$dW^{[2]} = dz^{[2]} a^{[1]T}$$

$$db^{[2]} = dz^{[2]}$$

$$dz^{[1]} = W^{[2]T} dz^{[2]} * g^{[1]'}(z^{[1]})$$

$$dW^{[1]} = dz^{[1]} x^T$$

$$db^{[1]} = dz^{[1]}$$

$$\underline{dz^{[2]}} = \underline{a^{[2]}} - \underline{y}$$

$$dW^{[2]} = dz^{[2]} a^{[1]T}$$

$$db^{[2]} = dz^{[2]}$$

$$\underset{(n^{[2]}, 1)}{dz^{[1]}} = W^{[2]T} dz^{[2]} * g^{[1]'}(z^{[1]})$$

$$dW^{[1]} = dz^{[1]} x^T$$

$$db^{[1]} = dz^{[1]}$$

Vectorized Implementation:

$$z^{[2]} = W^{[2]} x + b^{[2]}$$

$$a^{[2]} = g^{[2]}(z^{[2]})$$

$$z^{[2]} = \begin{bmatrix} z^{[2](1)} \\ z^{2} \\ \vdots \\ z^{[2](n)} \end{bmatrix}$$

$$z^{[2]} = W^{[2]} X + b^{[2]}$$

$$A^{[2]} = g^{[2]}(z^{[2]})$$

$$\underline{dZ^{[2]}} = \underline{A^{[2]}} - \underline{Y}$$

$$J(\cdot) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}_i, y_i)$$

$$dW^{[2]} = \frac{1}{m} dZ^{[2]} A^{[1]T}$$

$$db^{[2]} = \frac{1}{m} \text{np.sum}(dZ^{[2]}, \text{axis} = 1, \text{keepdims} = \text{True})$$

$$\underset{(n^{[2]}, m)}{dZ^{[1]}} = \underset{(n^{[2]}, m)}{W^{[2]T} dZ^{[2]}} * \underset{(n^{[2]}, m)}{g^{[1]'}(Z^{[1]})}$$

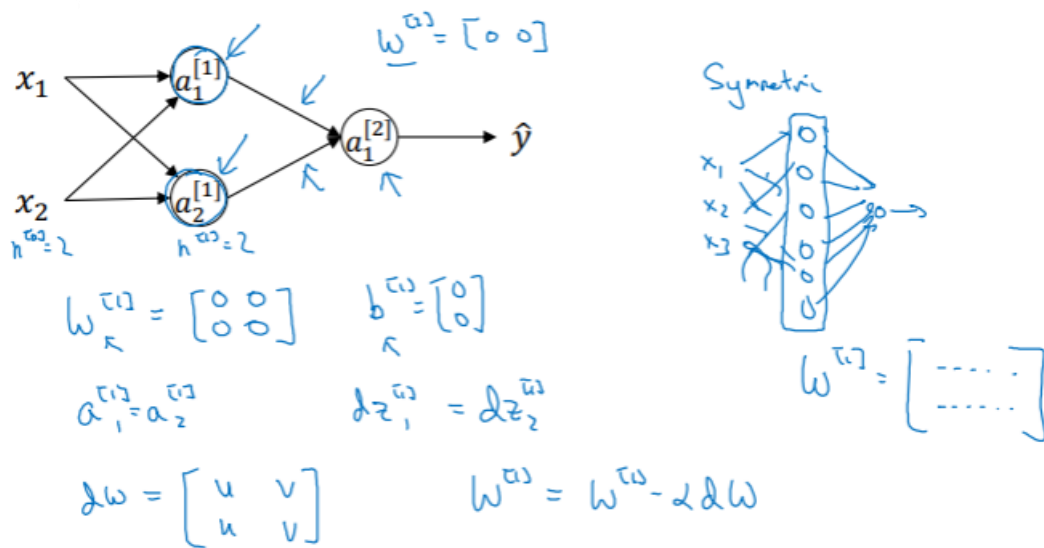
element-wise product

$$dW^{[1]} = \frac{1}{m} dZ^{[1]} X^T$$

$$db^{[1]} = \frac{1}{m} \text{np.sum}(dZ^{[1]}, \text{axis} = 1, \text{keepdims} = \text{True})$$

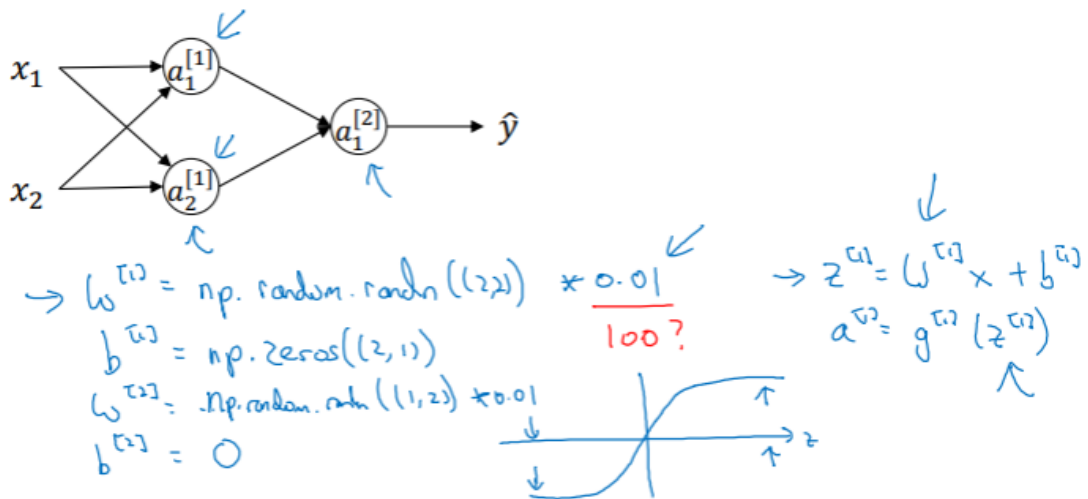
Random Initialization

- Initializing to zero
 - Initializing the bias terms b is okay, but not the w's
 - For any example you give it, a121 and a12 will be the equal because both of the hidden units are computing exactly the same function ("symmetric"). When you compute backpropagation, dz11 and dz12 will also be the same
 - Both hidden units start off computing the same function and have the same influence on the output unit



- Random initialization

- B does not have the symmetry breaking problem
- Initialize w's to small random values
- w too large $\rightarrow$ z be will large $\rightarrow$ end up at the ends of the sigmoid/tanh graph where the slope is small (less of an issue for other activation functions) $\rightarrow$ gradient descent is slow



Which of the following are true? (Choose all that apply.)

$W^{[1]}$ is a matrix with rows equal to the parameter vectors of the first layer.

$w_3^{[4]}$ is the row vector of parameters of the fourth layer and third neuron.

$w_3^{[4]}$ is the column vector of parameters of the third layer and fourth neuron.

W_1 is a matrix with rows equal to the parameter vectors of the first layer.

$w_3^{[4]}$ is the column vector of parameters of the fourth layer and third neuron.

$W^{[1]}$ is a matrix with rows equal to the transpose of the parameter vectors of the first layer.

The sigmoid function is only mentioned as an activation function for historical reasons. The tanh is always preferred without exceptions in all the layers of a Neural Network.

True/False?

False True

Which of these is a correct vectorized implementation of forward propagation for layer l , where $1 \leq l \leq L$?

$$Z^{[l]} = W^{[l]}A^{[l-1]} + b^{[l]}$$

$$Z^{[l]} = W^{[l-1]}A^{[l]} + b^{[l-1]}$$

$$A^{[l]} = g^{[l]}(Z^{[l]})$$

$$A^{[l]} = g^{[l]}(Z^{[l]})$$

$$Z^{[l]} = W^{[l]}A^{[l]} + b^{[l]}$$

$$Z^{[l]} = W^{[l]}A^{[l]} + b^{[l]}$$

$$A^{[l+1]} = g^{[l+1]}(Z^{[l]})$$

$$A^{[l+1]} = g^{[l]}(Z^{[l]})$$

The use of the ReLU activation function is becoming more rare because the ReLU function has no derivative for $c = 0$. True/False?

True **False**

Consider the following code:

```
#+begin_src python
x = np.random.rand(3, 2)
y = np.sum(x, axis=0, keepdims=True)
#+end_src
```

What will be `y.shape`?

(3,)

(3, 1)

(2,)

(1, 2)

Suppose you have built a neural network. You decide to initialize the weights and biases to be zero. Which of the following statements is true?

Each neuron in the first hidden layer will perform the same computation in the first iteration. But after one iteration of gradient descent, they will learn to compute different things because we have “broken symmetry”.

Each neuron in the first hidden layer will compute the same thing, but neurons in different layers will compute different things, thus we have accomplished “symmetry breaking” as described in the lecture.

Each neuron in the first hidden layer will perform the same computation. So even after multiple iterations of gradient descent, each neuron in the layer will be computing the same thing as other neurons.

The first hidden layer’s neurons will perform different computations from each other even in the first iteration; their parameters will thus keep evolving in their own way.

Logistic regression's weights w should be initialized randomly rather than to all zeros, because if you initialize to all zeros, then logistic regression will fail to learn a useful decision boundary because it will fail to "break symmetry", True/False?

True

False

Which of the following is true about the tanh function?

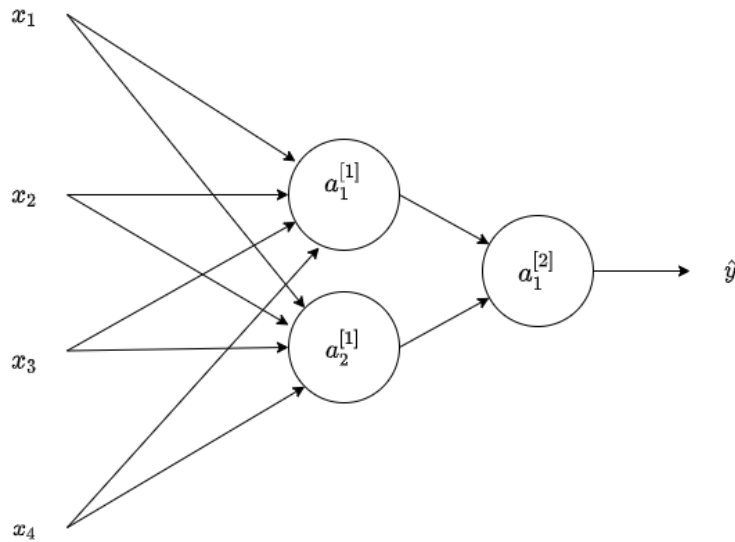
For large values the slope is close to zero. [making gradient descent slow]

For large values the slope is larger.

The derivative at $c = 0$ is not well defined.

The slope is zero for negative values.

Consider the following 1 hidden layer neural network:



Which of the following statements are True? (Choose all that apply).

$W^{[1]}$ will have shape (4, 2).

$W^{[2]}$ will have shape (1, 2)

$b^{[1]}$ will have shape (2, 1).

$b^{[1]}$ will have shape (4, 2)

$W^{[2]}$ will have shape (2, 1)

$W^{[1]}$ will have shape (2, 4).

Input layer $n^{[0]} = 4$

Hidden layer $n^{[1]} = 2$

Output layer $n^{[1]} = 1$

$W^{[l]}: (n^{[l]}, n^{[l-1]})$

$b^{[l]}: (n^{[l]}, 1)$

$W^{[1]}: (n^{[1]}, n^{[0]}) = (2, 4)$

$b^{[1]}: (n^{[1]}, 1) = (2, 1)$

$W^{[2]}: (n^{[2]}, n^{[1]}) = (1, 2)$

$b^{[2]}: (n^{[2]}, 1) = (1, 1)$

What are the dimensions of $Z^{[1]}$ and $A^{[1]}$?

$Z^{[1]}$ and $A^{[1]}$ are (4, m)

$Z^{[1]}$ and $A^{[1]}$ are (2, m)

$Z^{[1]}$ and $A^{[1]}$ are (2, 1)

$Z^{[1]}$ and $A^{[1]}$ are (4, 1)

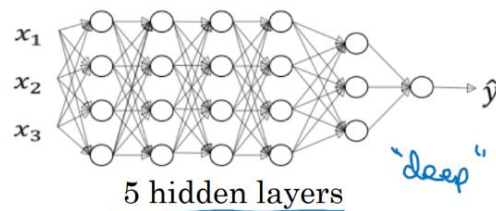
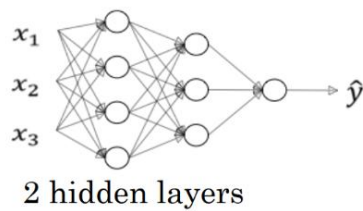
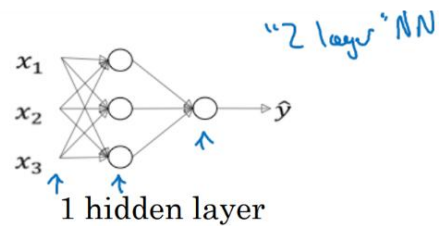
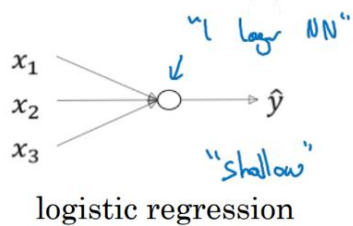
$Z^{[l]} = W^{[l]}A^{[l-1]} + b^{[l]}$ $A^{[l]} = g(Z^{[l]})$ $A^{[0]}: (4 \text{ features, } m \text{ training examples})$

$Z^{[1]} = W^{[1]}A^{[0]} + b^{[1]}: (2, 4) \times (4, m) + (2, 1) \rightarrow (2, m) + (2, 1) \rightarrow (2, m)$

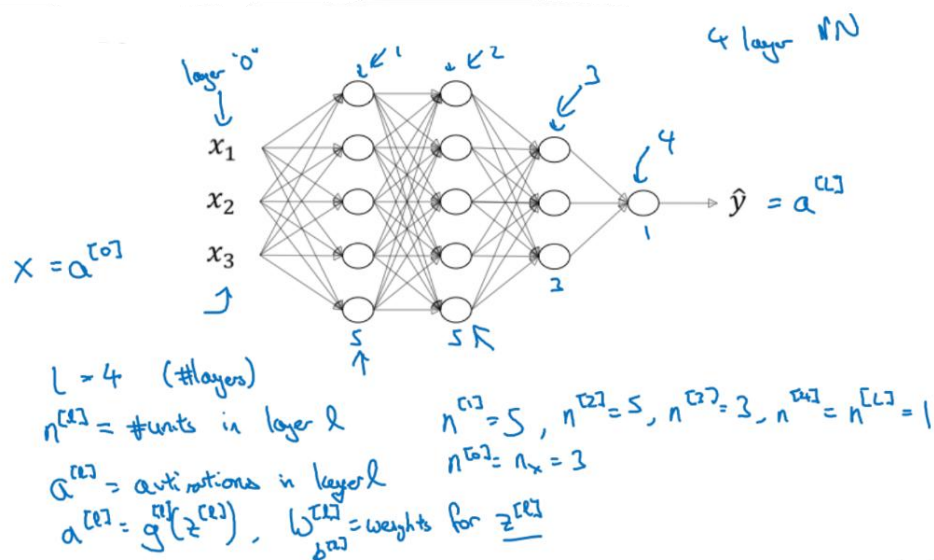
$A^{[1]} = g(Z^{[1]}) = (2, m)$ [applying the activation function doesn't change the dimensions]

Deep L-Layer Neural Networks

- What is a deep neural network?
 - Logistic regression is a shallow model (1 layer NN)
 - When counting layers L , do not include input. Only hidden and output

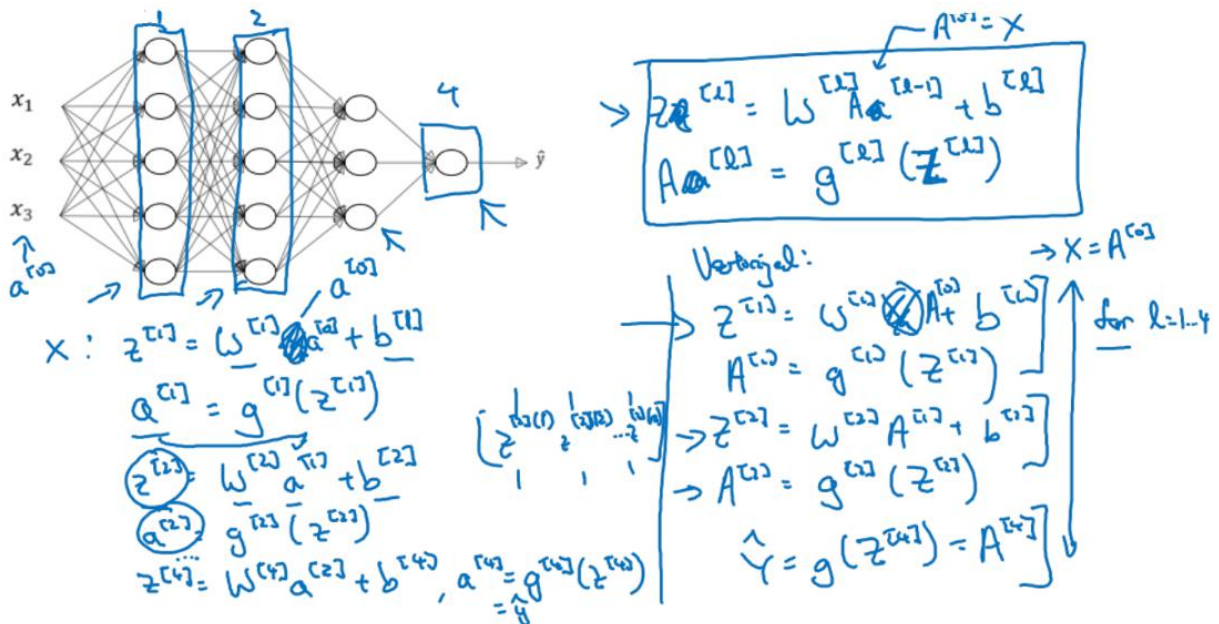


- Deep Neural Network Notation
 - $n^{[l]}$ = number of nodes in layer l
 - $a^{[l]}$ = activations in layer l
 - $w^{[l]}$ = weights for computing the value $z^{[l]}$ in layer l
 - x = input features = $a^{[0]}$
 - L = number of (non-input) layers



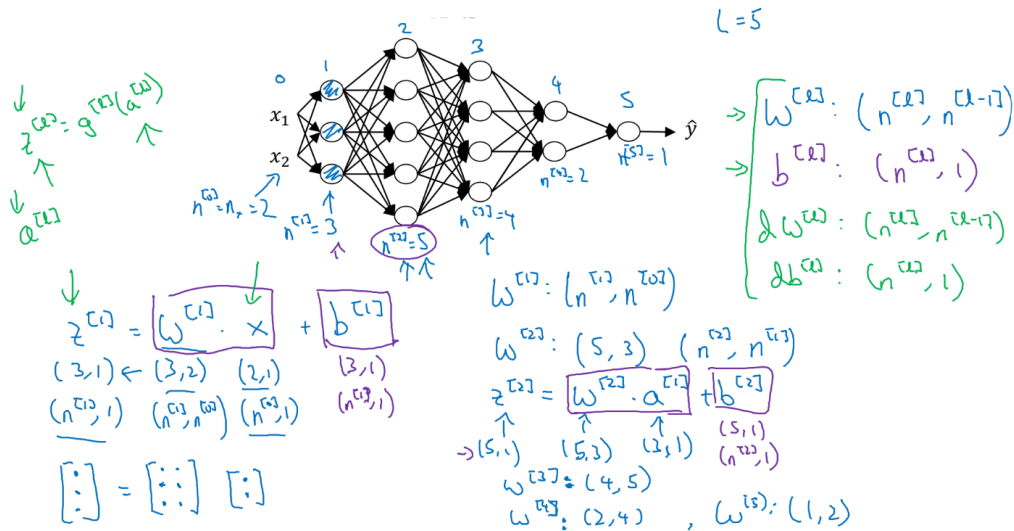
Forward Propagation in a Deep NN

- Single training example
 - $z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$ [parameters for layer l times the activations from the previous layer plus the bias vector]
 - $a^{[l]} = g^{[l]}(z^{[l]})$ [activation function applied to the values of z]
 - $\hat{y} = a^{[L]} = g^{[L]}(z^{[L]})$
 - Steps:
 - Compute $z^{[1]} = W^{[1]}x + b^{[1]}$ and $a^{[1]} = g^{[1]}(z^{[1]})$,
 - Then $z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$ and $a^{[2]} = g^{[2]}(z^{[2]})$, ...
 - $z^{[4]} = W^{[4]}a^{[3]} + b^{[4]}$ and $a^{[4]} = g^{[4]}(z^{[4]})$
 - $x = a^{[0]}$ because the input features x is also the activations of layer zero
- Whole training set
 - $Z^{[l]} = W^{[l]}A^{[l-1]} + b^{[l]}$, $A^{[l]} = g^{[l]}(Z^{[l]}) \rightarrow \hat{Y} = g(Z^{[L]}) = A^{[L]}$

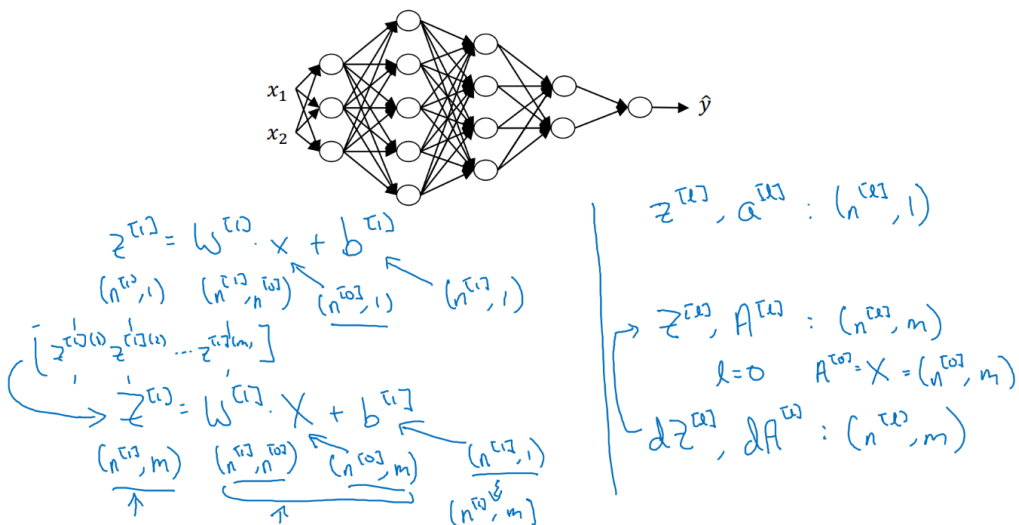


Matrix Dimensions

- Parameters $W^{[l]}$ and $b^{[l]}$
 - Verify matrix dimensions for your NN
 - $z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]} \rightarrow (n^{[l]}, 1) = (n^{[l]}, n^{[l-1]}) * (n^{[l-1]}, 1) + (n^{[l]}, 1)$
 - $z^{[l]}, a^{[l-1]}: (n^{[l]}, 1)$
 - $W^{[l]}$ (and dW): $(n^{[l]}, n^{[l-1]})$ $b^{[l]}$ (and db): $(n^{[l]}, 1)$
 - $\dim(dW) = \dim(W)$ and $\dim(db) = \dim(b)$ [relevant to backprop]

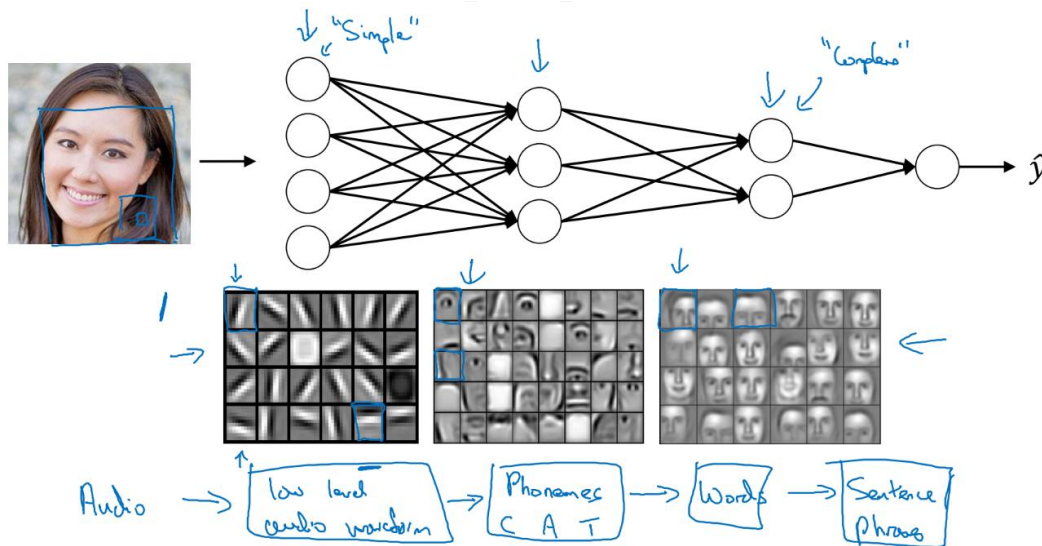


- Vectorized implementation
 - X now has m training examples and $Z^{[l]}$ is now $(n^{[l]}, m)$
 - $Z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]} \rightarrow (n^{[l]}, m) = (n^{[l]}, n^{[l-1]}) * (n^{[l-1]}, m) + (n^{[l]}, 1)$ [broadcast $\rightarrow (n^{[l]}, m)$]
 - $Z^{[l]}, A^{[l]}$ (and dZ, dA): $(n^{[l]}, m)$

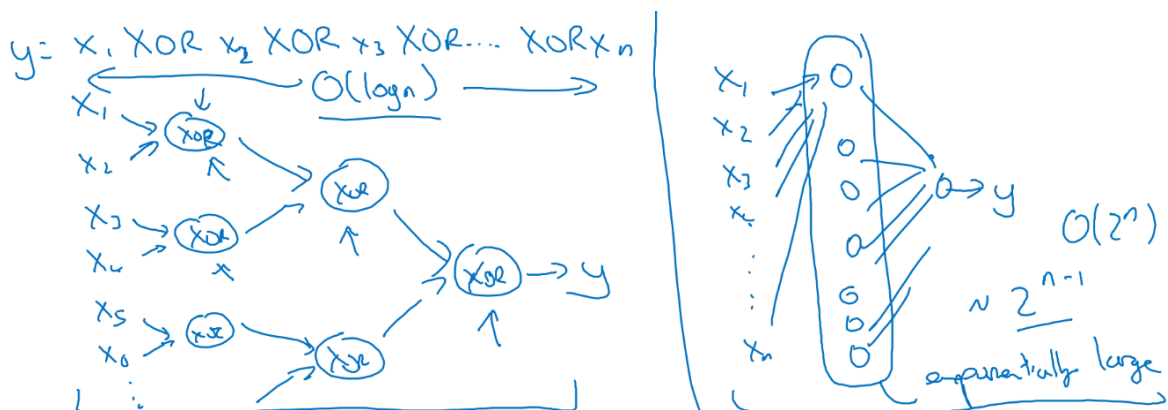


Advantages of Deep Representations

- Find simple things (like edges) and build them up and composing them to detect more complex things (like an eye or nose), then composing those to find even more complex things (like face)
- Sound recognition: pitch up or down, white noise, etc → phenomes → words → sentences

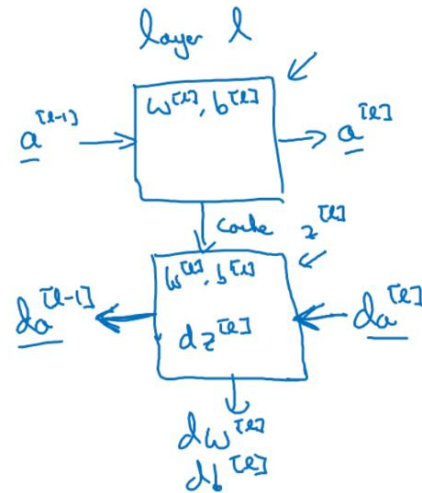
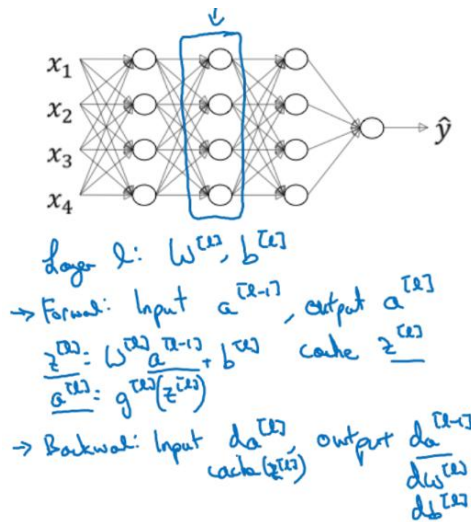


- Circuit theory and deep learning:
 - There are functions you can **compute with a "small"** (small # hidden units) **L-layer deep neural network** that **shallower networks require exponentially more hidden units** to compute



Building Blocks of Deep Neural Networks

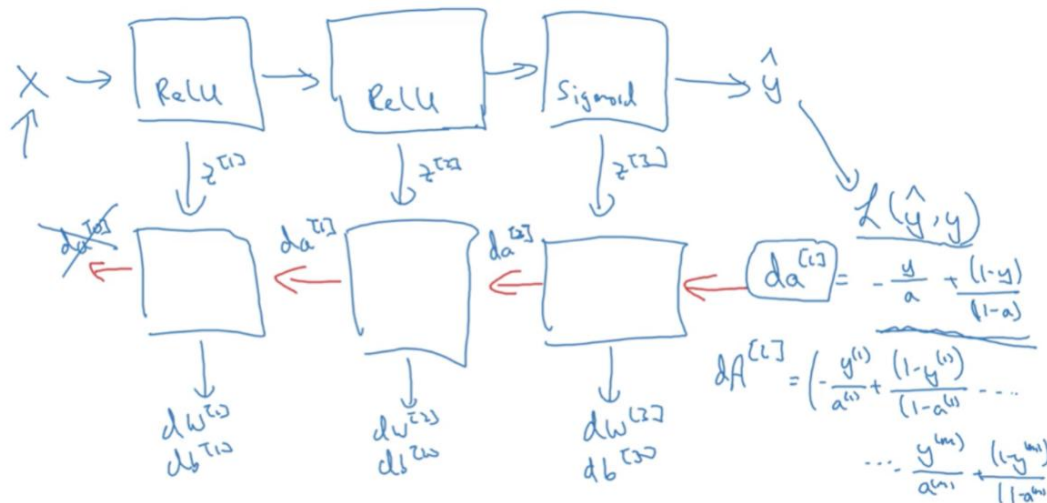
- In layer l
 - **Forward prop:** input $a^{[l-1]} \rightarrow$ forward function (computation using $\rightarrow$ output $a^{[l]}$ and cache containing $z^{[l]}$)
 - **Back prop:** input $da^{[l]} \rightarrow$ backprop function (computation using $W^{[l]}$ and $b^{[l]}$, calculates $dz^{[l]} \rightarrow da^{[l-1]}$, $dw^{[l]}$, and $db^{[l]}$. $da^{[l-1]} \leftarrow$ backward function $\leftarrow z^{[l]}$)
 - Tells you, given the derivatives with respect to these activations ($da^{[l]}$), what are the derivatives of $a^{[l-1]}$? How much does $a^{[l-1]}$ change the computed derivatives with respect to the activations from the previous layer?



- Basic computation of the NN
 - **Forward step:** input features $a^{[0]}$ feed into the forward prop function. Fxn computes the first layer $a^{[1]}$ (needs $W^{[1]}$ and $b^{[1]}$), and $z^{[1]}$, $W^{[L]}$, and $b^{[L]}$ are cached away. Now feed $a^{[1]}$ into the second layer, and then using $W^{[2]}$ and $b^{[2]}$, compute the activations $a^{[2]}$ in the next layer (caching $z^{[1]}$) and so on until you output $a^{[L]} = \hat{y}$.
 - **Backward step:** feed $da^{[L]}$ into the backward function (uses $W^{[L]}$ and $b^{[L]}$), which computes $dz^{[L]}$ and outputs $da^{[L-1]}$, $dw^{[L]}$, and $db^{[L]}$. Iterates backwards until we get $da^{[1]}$ ($da^{[0]}$ is the derivative wrt features, not useful for training weights of NN)
 - Cache during forward step stores the value of $z^{[l]}$ for the backward functions. The cache is a convenient way to get the parameters $W^{[L]}$ and $b^{[L]}$

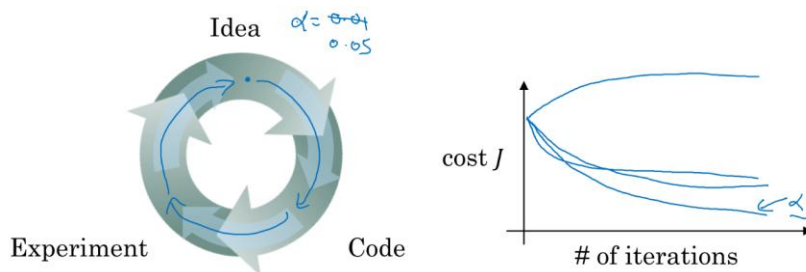
- Summary

- For logistic regression: initialize $da^{[L]} = -y/a + (1-y)/(1-a)$
- Vectorized version: $dA^{[L]} = [-y^{[1]}/a^{[1]} + (1-y^{[1]})/(1-a^{[1]}) \dots -y^{[m]}/a^{[m]} + (1-y^{[m]})/(1-a^{[m]})]$



Parameters vs Hyperparameters

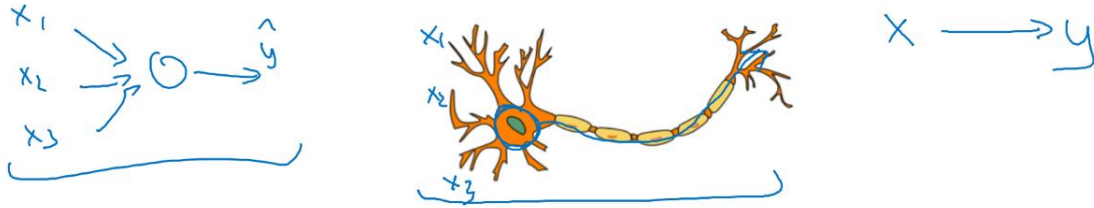
- Parameters: $W^{[1]}, b^{[1]}, W^{[2]}, b^{[2]}, \dots$
- Hyperparameters: learning rate (α), # iterations (i), # hidden layers (L), # hidden units ($n^{[1]}, n^{[2]}, \dots$), choice of activation function, later: momentum term, mini-batch size, regularization parameters, etc.
- Hyperparameters control the parameters
- Applied deep learning is empirical; you often just need to try out values (systematic ways in next module)



- Ex: Vision, speech, NLP (natural language processing), ads, web search, product recommendations
- Sometimes intuition of hyperparameters carries over, sometimes it doesn't

Brain Likeness

- A neuron sees electric signals from other neurons and fires its own signal. That's pretty much where the likeness stops.



$$dZ^{[L]} = A^{[L]} - Y$$

$$dW^{[L]} = \frac{1}{m} dZ^{[L]} A^{[L-1]T}$$

$$db^{[L]} = \frac{1}{m} np.sum(dZ^{[L]}, axis = 1, keepdims = True)$$

$$dZ^{[L-1]} = W^{[L]T} dZ^{[L]} * g'^{[L-1]}(Z^{[L-1]})$$

Note that * denotes element-wise multiplication)

⋮

$$dZ^{[1]} = W^{[2]T} dZ^{[2]} * g'^{[1]}(Z^{[1]})$$

$$dW^{[1]} = \frac{1}{m} dZ^{[1]} A^{[0]T}$$

Note that $A^{[0]T}$ is another way to denote the input features, which is also written as X^T

$$db^{[1]} = \frac{1}{m} np.sum(dZ^{[1]}, axis = 1, keepdims = True)$$

We use the "cache" in our implementation of forward and backward propagation to pass useful values to the next layer in the forward propagation. True/False?

False

True

useful to backward propagation

During the backpropagation process, we use gradient descent to change the hyperparameters. True/False?

False

True

changes the parameters (weights and biases)

Which of the following statements is true?

The deeper layers of a neural network are typically computing more complex features of the input than the earlier layers.

The earlier layers of a neural network are typically computing more complex features of the input than the deeper layers.

Vectorization allows us to compute $a^{[l]}$ for all the examples on a batch at the same time without using a for loop. True/False?

False

True

Assume we store the values for $n^{[l]}$ in an array called `layer_dims`, as follows: `layer_dims = [nx, 4,3,2,1]`. So layer 1 has four hidden units, layer 2 has 3 hidden units and so on. Which of the following for-loops will allow you to initialize the parameters for the model?

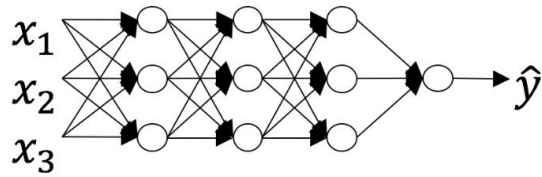
```
for i in range(1, len(layer_dims)):
    parameter['W' + str(i)] = np.random.randn(layer_dims[i-1], layer_dims[i]) * 0.01
    parameter['b' + str(i)] = np.random.randn(layer_dims[i], 1) * 0.01
```

```
for i in range(1, len(layer_dims)/2):
    parameter['W' + str(i)] = np.random.randn(layer_dims[i], layer_dims[i-1]) * 0.01
    parameter['b' + str(i)] = np.random.randn(layer_dims[i], 1) * 0.01
```

```
for i in range(1, len(layer_dims)/2):
    parameter['W' + str(i)] = np.random.randn(layer_dims[i], layer_dims[i-1]) * 0.01
    parameter['b' + str(i)] = np.random.randn(layer_dims[i-1], 1) * 0.01
```

```
for i in range(1, len(layer_dims)):
    parameter['W' + str(i)] = np.random.randn(layer_dims[i], layer_dims[i-1]) * 0.01
    parameter['b' + str(i)] = np.random.randn(layer_dims[i], 1) * 0.01
```

How many layers does this network have?



The number of layers L is 5. The number of hidden layers is 4.

The number of layers L is 4. The number of hidden layers is 4.

The number of layers L is 3. The number of hidden layers is 3.

The number of layers L is 4. The number of hidden layers is 3.

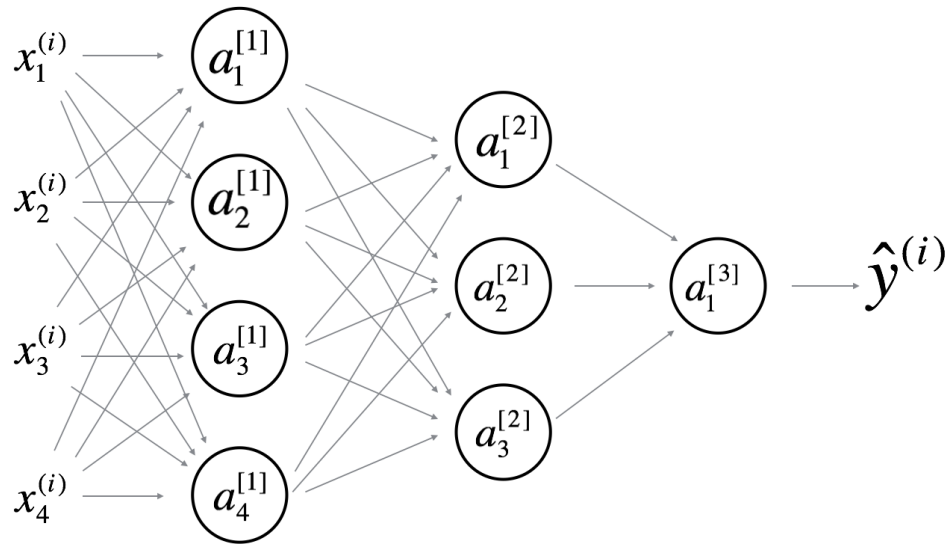
During forward propagation, in the forward function for a layer l you need to know what the activation function in a layer is (sigmoid, tanh, ReLU, etc.). During backpropagation, the corresponding backward function also needs to know what the activation function for layer l is, since the gradient depends on it. True/False?

False **True**

A shallow neural network with a single hidden layer and 6 hidden units can compute any function that a neural network with 2 hidden layers and 6 hidden units can compute. True/False?

False True

Consider the following 2 hidden layer neural network:



Which of the following statements are True? (Choose all that apply).

$b^{[1]}$ will have shape (4, 1)

$b^{[1]}$ will have shape (3, 1)

$b^{[1]}: (n^{[1]}, 1) = (4, 1)$

$b^{[2]}$ will have shape (3, 1)

$b^{[2]}$ will have shape (1, 1)

$b^{[2]}: (n^{[2]}, 1) = (3, 1)$

$b^{[3]}$ will have shape (3, 1)

$b^{[3]}$ will have shape (1, 1)

$b^{[3]}: (n^{[3]}, 1) = (1, 1)$

$W^{[1]}$ will have shape (3, 4)

$W^{[1]}$ will have shape (4, 4)

$W^{[1]}: (n^{[1]}, n^{[0]}) = (4, 4)$

$W^{[2]}$ will have shape (3, 4)

$W^{[2]}$ will have shape (3, 1)

$W^{[2]}: (n^{[2]}, n^{[1]}) = (3, 4)$

$W^{[3]}$ will have shape (3, 1)

$W^{[3]}$ will have shape (1, 3)

$W^{[3]}: (n^{[3]}, n^{[2]}) = (1, 3)$

$n^{[0]} = 4, n^{[1]} = 4, n^{[2]} = 3, n^{[3]} = 1$

Whereas the previous question used a specific network, in the general case what is the dimension of $b^{[l]}$, the bias vector associated with layer l ?

$b^{[l]}$ has shape $(1, n^{[l]})$

$b^{[l]}$ has shape $(n^{[l+1]}, 1)$

$b^{[l]}$ has shape $(1, n^{[l-1]})$

$b^{[l]}$ has shape $(n^{[l]}, 1)$